多级系统在线学习笔记

全信息&全share&DNEG的区别

全信息：最后一层的cost都知道

全share：只有抽到share的子节点它才会把cost传递给所有它的父节点

DNEG：节点i知道所有子节点j的cost

Agent调研学习

全share算法内容

叶子节点维护 θ

我们可以把更新过程分为两个阶段：**1. 叶子节点的自我更新** 和 **2. 祖先节点的聚合更新**。

第一阶段：叶子节点的自我更新 (Leaf Update)

当算法选中一个叶子节点 ℓ_t 并观察到真实代价 c_t 后，该叶子节点会调整自己的权重。

1. 更新累积得分 (theta)

首先，我们更新该叶子的对数权重（也叫势能/得分，Potential）：

$$\theta_{\ell_t}^{(t+1)} = \theta_{\ell_t}^{(t)} - \eta \cdot c_t$$

- η **(学习率)**：步长。
- c_t **(真实代价)**：注意！这里直接减去 c_t，**没有除以概率** p_t 。
  - 这就是 Full-Share 的核心：我们假设对这条路径拥有“全信息”，直接给予惩罚。
  - 如果 c_t 很大（表现差），θ 就会大幅减小（变得更负）。

2. 更新指数权重 (w)

将得分转化为非负的权重：

$$w_{\ell_t}^{(t+1)} = \exp\left(\theta_{\ell_t}^{(t+1)}\right)$$

这在数学上等价于乘法更新：

$$w_{\ell_t}^{(t+1)} = w_{\ell_t}^{(t)} \cdot e^{-\eta c_t}$$

- **物理意义**：权重会按指数级衰减。
  - 如果 $c_t > 0$，则 $e^{-\eta c_t} < 1$，权重 w 变小。
  - 代价越大，衰减越狠。

第二阶段：祖先节点的聚合更新 (Ancestors Update / Sharing)

这是树结构特有的步骤。叶子变了，它的所有“长辈”（父节点、祖父节点...）维护的**子树总权重 (S)** 也必须跟着变。

1. 计算变化量 (Δw)

我们要算出这个叶子的权重到底变了多少：

$$\Delta w = w_{\ell_t}^{(t+1)} - w_{\ell_t}^{(t)}$$

- 通常情况下，因为 Cost 是正的，权重会减小，所以 Δw 是一个**负数**。

2. 向上回传 (Back-propagation)

找到从该叶子 ℓ_t 到根节点路径上的所有边。对于路径上的每一对父子关系 (u, v)（u 是父，v 是子），更新它们对应的聚合权重 $S[u, v]$：

$$S[u, v] \leftarrow S[u, v] + \Delta w$$

- **注意**：$S[u, v]$ 本来存储的是“以 v 为根的子树中所有叶子的权重之和”。
- 因为其中一个叶子少了 Δw，所以总和也自然少了 Δw。
- **不需要重新遍历整棵树**，只需要更新这一条路径上的 L 个节点。

为什么w初始化不能全设为常数

1. 为什么初始化必须是叶子总和?

正常情况（Line 4 正确初始化）：

- 初始时 $\theta = 0, w = 1$。
- $W[u]$ 初始化为子树叶子数量。
- 此后每次更新 $W \leftarrow W + \Delta$，数学归纳法保证了 $W[u]$ 永远等于 $\sum w_\ell$。

异常情况（初始化为常数 C）：

假设你把所有 $W[u]$ 强行初始化为常数 C（比如 $C = 1$）。

- 真实的初始权重和是 $S_{true} = |\text{Leaves}(u)|$。
- 你引入了一个**永久性的加性误差**：$\text{Bias}_u = C - S_{true}$。

因为代码中的更新逻辑是 **增量更新**（$W \leftarrow W + \Delta$），这个初始的误差 Bias_u **永远不会被消除**。

在第 t 轮，你的 $W[u]_t$ 实际上变成了：

$$W[u]_t = \left(\sum_{\ell \in \text{subtree}(u)} e^{\eta \theta_{\ell,t}} \right) + \text{Bias}_u$$

2. 当 $t \to \infty$ 时会发生什么?

随着 Cost 不断累积（假设 Cost 总是正的），θ 会变得越来越负，因此**真实的权重项 $e^{\eta\theta}$ 会趋近于 0**。

让我们看看概率公式会变成什么样：

$$P(j|i) = \frac{W[j]_t}{W[i]_t} = \frac{\text{真实权重和}_j + \text{Bias}_j}{\text{真实权重和}_i + \text{Bias}_i}$$

当 $t \to \infty$ 时：

- **分子**：真实权重和$_j \to 0$
- **分母**：真实权重和$_i \to 0$

于是，选择概率极限为：

$$\lim_{t\to\infty} P(j|i) = \frac{0 + \text{Bias}_j}{0 + \text{Bias}_i} = \frac{\text{Bias}_j}{\text{Bias}_i}$$

结论：

算法最终的选择概率完全取决于你**初始化的那个常数带来的误差**，而**与叶子节点的真实表现（Cost）完全无关**！

1. 为什么“增量回传”必须依赖正确的初始值？

你的算法逻辑是：

1. **选择**：$P(j|i) = \frac{W[j]}{W[i]}$
2. **更新**：算出叶子权重的变化量 $\Delta = w_{new} - w_{old}$
3. **回传**：$W[ancestor] \leftarrow W[ancestor] + \Delta$

这意味着，$W[u]$ 在任意时刻 t 的值等于：

$$W[u]_t = \text{初始值} + \sum \text{历史所有的变化量}$$

数学崩塌推演：

假设节点 u 下面有 100 个叶子。

- **正确的数学事实**：这 100 个叶子的初始权重都是 1，总和应为 100。如果它们最后都衰减到 0，那么总的变化量 $\sum \Delta$ 将会是 -100。
- **错误的初始化**：你把 $W[u]$ 强行初始化为 **1**。
- **后果**：随着时间推移，叶子不断被选中、扣分、权重衰减。当所有叶子的权重都趋近于 0 时，累积的 Δ 会趋近于 -100。
- **最终状态**：
  $$W[u]_{final} = 1(\text{初始}) + (-100)(\text{变化量}) = \mathbf{-99}$$

结果： $W[u]$ 变成了负数。

下一步选择时： $P = \frac{W[j]}{W[i]}$ 出现负数概率，**代码直接报错或逻辑崩盘**。

---

2. 为什么“只更新被选中的路径”救不了它？

你可能会想：“既然我只更新被选中的路径，没选中的就不变，是不是误差就不会扩散？”

恰恰相反，这会让选择偏差（Selection Bias）永久固化。

假设父节点有 A、B 两个分支：

- **分支 A**：有 100 个叶子（潜力大，容错率高）。
- **分支 B**：有 1 个叶子。

正确初始化：

- $W[A] = 100, W[B] = 1$。
- 算法有 $100/101 \approx 99\%$ 的概率选 A。这是合理的，因为 A 包含更多可能性。

$$\lim_{t \to \infty} P(j|i) = \frac{0 + \te\cdots$$

用户001126 1月27日 15:13

主要是这个原因不能换 w 初始化

错误初始化（设常数 1）：

- $W[A] = 1, W[B] = 1$。
- 算法有 50% 的概率选 A，50% 的概率选 B。
- **后果**：
  a. **起跑线错误**：算法严重低估了分支 A 的重要性（本来该经常选它，现在变成对半开）。
  b. **无法纠正**：因为没选中的分支不更新，所以如果 A 一直没被选中，它的 $W[A]$ 就一直是 1，永远无法变回它应有的 100。
  c. **越走越偏**：如果 B 分支稍微运气好一点，没怎么扣分，算法可能会一直以为 A 和 B 是一样重要的，从而在 B 上浪费大量时间（Regret 变大），或者因为 A 的某次扣分导致 $W[A]$ 变成负数（如前所述）。

3. 结论

$W[u]$ 的物理含义必须是“子树所有叶子的权重之和”。

- 如果初始化为常数，$W[u]$ 就失去了这个物理含义，变成了一个毫无意义的数字。
- **增量更新 (Δ)** 的前提是 **基准值 (Base)** 必须正确。基准错了，加减多少次 Δ 都是错的。

所以，即使对于你的算法，$W[u]$ 必须初始化为 `sum(exp(eta * theta))`（在 t=0 时即为叶子数量），绝对不能设为常数。

Risky下界

1) 构造：risky 链 + 每层一个“完整 shared 子树”侧枝（体现 share 的作用）

取整数 $R \geq 2$。构造一棵树 $\mathcal{T}$ 如下：

- risky 链节点：$v_1, v_2, \ldots, v_R$，其中 v_1 是 root。
- 对每个 $i = 1, \ldots, R-1$，节点 v_i 有两个孩子：
  1. **一个完整的 shared 子树** S_i：其所有叶子都属于 $\backslash\mathrm{Shared}$（因此 S_i 的根节点以及其内部节点全部是 Safe）
  2. 下一层 risky 节点 v_{i+1}
- 最底端节点 v_R 的两个孩子是两个叶子 u 与 u'，都属于 $\backslash\mathrm{Unshared}$。

于是：

- 任一路径一旦进入某个 S_i，后续全是 safe（因为子树全 shared），不可能再遇到 risky ——符合你定义“safe 后不可能再 risky”；
- 从 root 沿 risky 链 $v_1 \to v_2 \to \cdots \to v_R$ 连续遇到 risky 节点 **恰好** R **层**；
- 因此这棵树的“最长 risky 前缀长度”就是你定义的 R。

---

2) 代价分布设置（让每个 shared 子树“等价于一个固定均值的孩子”）

所有代价都在 $[0,1]$，使用 Bernoulli 代价（与原文一致）：

- 任何叶子 ℓ 被访问时产生代价 $c[\ell, t] \in \{0,1\}$，且 $\mathbb{P}(c[\ell,t]=1) = p_\ell$。

2.1 底层两个 unshared 叶子（制造 bandit 难度）

取

$$0 < \delta < \frac{1}{2^R}.$$

定义最优均值

$$p^{\backslash *} := \frac{1 - 2^R\delta}{2}.$$

令底层两叶 u, u' 满足

$$\min\{p_u, p_{u'}\} = p^{\backslash *}, \qquad \max\{p_u, p_{u'}\} = \frac{1 + 2^R\delta}{2}.$$

（即一好一坏、差距极小；这保证底层需要样本“教育”。）

[图片]

用户001126 3月12日 15:29

和原文唯一区别

2.2 每一层 shared 子树 S_i 的均值（体现“完整 shared 子树”）

让每个 shared 子树 S_i 内的**所有叶子**均为同分布 Bernoulli，均值设为

$$p_{S_i} := \frac{1-(2^R-2^{i-1})\delta}{2}.$$

也就是：对任意 $\ell \in \backslash\mathrm{Leaf}(S_i)$，都设 $p_\ell = p_{S_i}$。

于是对任意时刻 t，无论 S_i 内部怎么走（它是 full-share、全信息），都有

$$\mathbb{E}[y[S_i,t]] = p_{S_i}.$$

这点至关重要：**你把“侧枝叶子”升级成“完整 shared 子树”后，对父节点来说它仍等价于一个固定期望代价的孩子**，原文 proof 可以逐行复用。

特别地，

$$p_{S_1} = \frac{1-(2^R-1)\delta}{2} = p^{\backslash *} + \frac{\delta}{2}. \tag{Gap}$$

---

3) Regret 定义（对应原文 Eq.(4)）

令 $y[v_1,t]$ 为第 t 轮从 root 出发最终产生的代价。最优叶子的累计期望代价为 $p^{\backslash *}T$。因此 regret 为

$$\mathrm{Reg}(T) = \sum_{t=1}^{T}\left(\mathbb{E}[y[v_1,t]] - p^{\backslash *}\right) = \sum_{t=1}^{T}\left(\mathbb{E}[y[v_1,t]] - \frac{1-2^R\delta}{2}\right). \tag{4R}$$

原文regret下界思路

1）先构造一个“多层链条 + 侧枝叶子”的坏例子

他们搭了一个深度 $L+1$ 的系统：
从根往下是一条链 $1 \to 2 \to \cdots \to L$，每一层旁边都有一个“侧枝叶子”。最底层 L 下面有两个非常接近的叶子（谁更好差得极小），必须靠 bandit 学很多样本才能分清。

2）说明：底层没学会之前，系统每轮至少亏一个固定 gap（教育前阶段）

底层 bandit 在收到的样本次数不够前，期望表现一定不够好（Lemma 5）。
于是只要“教育还没完成”（底层收到任务次数 < 某个阈值），根节点每一轮的期望损失相对最优至少多 $\delta/2$。
→ 所以“教育前”每轮都有固定 regret。

3）关键：教育到底要多久？因为要穿过 $L-1$ 层概率连乘（教育很慢）

想让任务到达底层 L，必须每一层都选择“继续往下”的孩子。
每层最多以概率 q_i 往下送，所以一轮到达 L 的概率最多 $\prod_{i=1}^{L-1} q_i$。
要让 L 收到足够多 N_δ 个样本，平均至少要

$$\mathbb{E}[T_\delta] \geq \frac{N_\delta}{\prod q_i}$$

轮。
→ 链越深，这个乘积越小，教育越慢，前半段 regret 越大。

4）教育完成后也不会“零 regret”，因为上层仍可能不把任务送到底（教育后阶段）

即使底层学好了，上层策略为了“短期不亏”，仍可能中途选侧枝叶子（不送到底层）。
只要没送到底层，就会落在侧枝叶子上，而侧枝叶子的期望代价被构造得至少比最优大 $\delta/2$。
于是教育后每轮的期望 regret 至少是

$$(\text{没送到底层的概率}) \times \frac{\delta}{2}.$$

5）把两段加起来：无论你怎么选 q_i，总 regret 至少是一个多项式量级

总下界长这样（Eq.(8)）：

- 前段：$\frac{\delta}{2} \cdot \mathbb{E}[T_\delta]$（教育慢导致）
- 后段：$\frac{\delta}{2} \cdot (T - \mathbb{E}[T_\delta]) \cdot (\text{不送到底层概率})$

这两项是对抗的：

- 你若让 q_i 很小（不愿教育）→ $\prod q_i$ 很小 → 教育时间爆炸 → 前段大；
- 你若让 q_i 很大（愿意教育）→ 不送到底层概率也会上去（大概 $\sim \sum q_i$）→ 后段大。

最优平衡大约取 $q_i \sim T^{-1/L}$，结果两段都会变成

$$T^{\frac{L-1}{L}},$$

所以得到整体下界

$$\mathrm{Reg}(T) = \Omega\left(T^{\frac{L-1}{L}}\right).$$

整体原则：算法只做“策略选择”，不做…

用户001126 3月25日 17:31 （编辑…

planner 在首轮建立树结构，cost 定义，share/unshare 定义

用户001126 3月25日 17:52

固定树里放的应该是：
任务约束下“可允许的高层候选策略”，

用户001126 3月25日 17:53

planner 少规划了怎么办？
这会发生。
这说明固定树覆盖不全，但不是说在线时要把错节点混进去。
正确处理是：
- 当前轮次里，少规划造成的失误先体现为某些合法 leaf 的高 cost
- 如果这种失败反复出现，说明 tree schema 不完整
- 下一版离线重新设计树

也就是：
单次在线学习时树固定；跨实验阶段可以修树。

用户001126 3月26日 10:38

@用户 001126 webpilot 中的 planner 只是 HTD 分解任务，并不会生成原子动作的完整树

用户001126 3月26日 10:45

@用户 001126 先生成一个高层的子任务分解 / 计划骨架
在 WebPilot 里，Planner 负责做 HTD（Hierarchical Task Decomposition），把复杂任务拆成多个可管理的 subtasks，目的是缩小后续搜索范围，而不是枚举所有原子动作。
在执行过程中不断修正这个高层计划
WebPilot 还有 RTA（Reflective Task Adjustment），也就是执行完一个 subtask 之后，Planner 和 Controller 会根据新观测再调整高层 plan，所以它不是“一次预测完最终正确树”，而是先给骨架，再边执行边改。

展开

算法应用agent环境背景（旧版web已弃用）

一、你的 Risky-PS 映射到 web agent 的统一说明

1. 整体原则：算法只做“策略选择”，不做底层执行

把 web agent 分成两层看：

- **执行层 agent**：负责看页面、点按钮、读内容、调用工具、完成具体网页操作
- **学习层 Risky-PS**：只负责在一棵**固定的高层策略树**上，选择本轮走哪条 root-to-leaf 策略

也就是说，真实 agent 内部怎么通信、怎么执行，是执行层的事；你的算法只看到：

- 固定树
- 每轮选到的 leaf
- leaf 执行结束后的一个标量 cost

这和你原算法接口一致：每轮从 root 一路走到 leaf，然后观察 leaf cost 再更新。

---

2. 树怎么定义：不是原子 DOM 树，而是固定的高层策略树

为了和你的 fixed-tree 背景一致，不能把“网页上实时出现的所有 click”直接当树。应该先把一类结构化 web task **编译成固定高层树**：

- **root**：当前任务模板
- **内部节点**：阶段性策略选择位点
- **边 / child**：某一类候选子策略
- **leaf**：一个完整的 terminal strategy / macro-policy

你自己的笔记已经把这种映射写出来了：web task 是树，中间节点是阶段性决策，叶节点是一条完整网页交互轨迹。

---

3. 叶子怎么定义：leaf 不是“当前动作”，而是“完整 terminal policy”

这是最关键的一点。

在你的算法里：

- 每轮是从 root 走到 leaf
- **只有到 leaf 才 observe cost**
- 更新也是 leaf update + ancestor aggregate update。

所以在 web 映射里，leaf 不能定义成“当前 click / 当前动作”，否则你就把问题改成 step-wise RL 了。

正确的是：

leaf = 当前固定树下的一条完整 terminal strategy

例如在退款表单任务里，一个 leaf 可以是：

- 查订单 → 提取金额 → 模板填表 → 停在 review

而不是“点开订单页”这种中间动作。

学习层 Risky-PS：只负责在一棵固定的…

用户001126 3月25日 17:19

这个可以作为当前节点的 agent 选择孩子的参考，占一定比重或者完全按这个来

cost 只在 leaf 执行结束后读一次。父节…

用户001126 3月25日 17:21

这个 cost 是执行 agent 计算的，生成后作为学习层的输入

4. cost 怎么定义：只在 leaf 处定义 terminal cost

cost 一定是 **leaf-level terminal cost**，不是中间节点 cost。

你自己的笔记已经给了标准写法：

当执行到某个叶节点后，环境根据任务完成情况、执行步数和其他任务相关指标，返回一个标量 cost，且可以归一到 ([0,1])。

所以在 web agent 里，一轮的 cost 可统一写成：

$c_t(\ell) \in [0, 1]$

它通常由几部分组成：

- 任务是否成功
- 是否达到 evaluator 要求的终态
- 代价项，例如步数、超时、明显偏航

重点是：
cost 只在 leaf 执行结束后读一次。父节点不生成自己的 cost，也不向下传 cost。

---

5. shared / unshared 到底是什么意思

这里一定要和“真实通信”分开。

shared / unshared 不是在说 agent 的真实信息能不能上传，
而是在说：leaf 的那个抽象 terminal cost，是否应该被解释成祖先前缀路径的公共监督。

shared leaf

如果某个 leaf 的终局 cost，主要在评价：

- 整条公共前缀路线是否正确
- 路由 / 筛选 / 判断 /导航是否合理

那么它就适合建模成 **shared**。
这时被访问后，叶子更新会产生一个增量 (\Delta)，并把这个 (\Delta) 作为 shared aggregate 的加法更新沿祖先路径回传。你笔记里也明确写了：被选中的 shared leaf 的 cost 会沿路径上传，但不是所有 shared leaves 的 cost 都知道。

unshared leaf

如果某个 leaf 的终局 cost，主要在评价：

- 最后这个 leaf-specific 的末端分支本身
- 一个私有末端执行臂选得对不对

那么它更适合建模成 **unshared**。
这时它不能被祖先当成 shared 的公共证据；它只能作为被访问样本的 bandit feedback，经 IPS 修正后更新该 leaf，再通过 unshared aggregate 的乘法因子 (r) 间接影响祖先。你原算法与笔记都把 unshared 看成 bandit 模式，并强调其无偏估计与方差项。

---

6. shared / unshared 的实质判据

最稳妥的建模判据就是一句话：

> 可以预先定义一个高层树：

用户001126 3月25日 17:45

如果有节点做了 planner 以外的行为，需要 controller 进行约束

某个 leaf cost 被建模为 shared，当且仅当我们希望把该终局反馈解释为对整条已实现前缀的公共监督；如果该终局反馈主要由 leaf-specific 的末端分支决定，就应建模为 unshared。

一个很实用的测试是：

- 固定前缀不变，只改最后一个 sibling leaf
- 如果 cost 的意义仍主要在评价共同前缀，则偏 shared
- 如果 cost 会本质改变，说明主要在评价最后私有末端臂，则偏 unshared

---

7. safe / risky 节点怎么定义

这里必须完全按你算法原定义来：

- **safe 节点**：后继子树全是 shared leaves
- **risky 节点**：后继子树里混有 unshared leaves。

这不是“动作危险不危险”的意思，而是 **feedback structure** 的意思。
你的理论也明确说明：safe 节点的探索成本可以降到 0，而 risky 节点才需要支付探索和 unshared variance 代价；结构优势取决于 unshared 的分布位置。

二、把这套说明落到webarena-verified-hard数据集里的 3 个例子

我选 3 个最合适的：

1. **全 share**：task 105
2. **半 share / mixed**：task 529
3. **近似全 unshared**：task 791

这样三种端点都有。

例子 1：task 105，GitLab issue 导航 —— 全 share

任务是：导航到 `OpenAPITools/openapi-generator` 仓库里，显示 “not yet closed” 且 label related to `OpenAPI Generator CLI` 的 issues 页。evaluator 检查最终 issues URL，以及 GraphQL referer 是否匹配对应筛选页。

Navigate to the page showing the list of not yet closed issues in the OpenAPITools/openapi-generator repository that have labels related to OpenAPI Generator CLI"

固定树示意

可以预先定义一个高层树：

- root: 完成 OpenAPITools/openapi-generator 的 issue 过滤导航任务
  - v_1: 先定位仓库，再进入 issues
    - u_1: 先设置 state 过滤
      - ℓ_1: 进入 openapi-generator → 打开 issues → 设为 not yet closed → 加 label=OpenAPI Generator CLI
      - ℓ_2: 进入 openapi-generator → 打开 issues → 设为 not yet closed → 加其他相关 label
    - u_2: 先设置 label 过滤
      - ℓ_3: 进入 openapi-generator → 打开 issues → 先加 label=OpenAPI Generator CLI → 再设为 not yet closed
      - ℓ_4: 进入 openapi-generator → 打开 issues → 先加其他相关 label → 再设为 not yet closed
  - v_2: 先进入 issues 总页，再筛 repo / filter
    - u_3: 通过页面控件逐步过滤
      - ℓ_5: 进入 issues 总页 → 切到目标 repo → 设为 not yet closed → 加 label=OpenAPI Generator CLI
      - ℓ_6: 进入 issues 总页 → 切到目标 repo → 设为 not yet closed → 加其他相关 label
    - u_4: 通过 URL / 查询参数过滤
      - ℓ_7: 进入 issues 总页 → 切到目标 repo → URL 参数设为 not yet closed + label=OpenAPI Generator CLI
      - ℓ_8: 进入 issues 总页 → 切到目标 repo → URL 参数设为 not yet closed + 其他 label

这个例子里，你可以把 $\ell_1, \ldots, \ell_8$ 都建成 **shared leaves**，所以整棵树接近 **全 share**。

- leaf：一条完整导航策略
  例如
  “进入 repo → 打开 issues → 设置 state=opened → 添加 label=OpenAPI Generator CLI”

cost

定义成一个导航型终局 cost，例如：

$$c_t(\ell) = 0.7 \cdot \mathbf{1}\text{最终页面不对} + 0.2 \cdot \text{normalized steps} + 0.1 \cdot \mathbf{1}\text{缺少预期 referer/filter 结果}$$

为什么是全 share

这里 terminal cost 基本都在评价：

- repo 路由是否正确
- issues 过滤路径是否正确
- 最终导航是否达到目标状态

它没有一个“最后私有末端臂”主导成败。
所以这类 leaf 的 cost 非常适合作为 **对公共前缀路径的监督**。

因此这个任务最自然建模为 **all-share / safe subtree**：

- 所有 leaf 都是 shared
- 所有内部节点都是 safe
- 被访问 leaf 更新后，祖先收到的是 shared 的 (\Delta)

这和你 all-share / safe subtree 的理论端点是对齐的。

例子 2：task 529，退款表单，停在 revie…

用户001126 3月25日 18:11

目前问题：

如果 planner 只设置正确的结果的树（而不是所有可行动作空间），那么结果层是否对整条链路有信息需要依靠叶子层的 cost 比重，但是 planner 无法预测 cost 比重。

例子 2：task 529，退款表单，停在 revie…

用户001126 3月26日 10:16

本地可检验 artifact leaves 设为 shared，对外生效 mutation leaves 设为 unshared。

例子 2：task 529，退款表单，停在 revie…

用户001126 3月26日 10:30

还有一个问题，感觉实现层面挺难统计 regret 的?

用户001126 3月26日 10:30

或者只能对比纯 agent 与加入我们算法的提升

例子 2：task 529，退款表单，停在 review，不要 submit —— 半 share / mixed

任务要求：

- 填 contact us form
- comment 中必须带正确订单号和金额
- evaluator 要看到 dummy_bin 的表单内容匹配
- 不要submit
- **Fill out the contact us form with this refund message for the Stereo Sound System bought Feb 2023: \"I need a refund. Order: <order_id>. Amount: <amount>. Reason: It broke after three days of use.\" Leave the filled form ready for review and do not submit it.**

固定树示意

- root: 完成退款表单任务并停在 review
  - v_1: 查订单信息
    - u_1: 自动生成 comment
      - ℓ_1: 查到正确订单 → 自动生成正确内容 → 停在 review
      - ℓ_2: 查到正确订单 → 自动生成正确内容 → submit
    - u_2: 模板填充 comment
      - ℓ_3: 查到正确订单 → 模板填对内容 → 停在 review
      - ℓ_4: 查到正确订单 → 模板填对内容 → submit
  - v_2: 不查订单，直接猜
    - u_3: 自动生成 comment
      - ℓ_5: 猜错订单 → 自动生成内容 → 停在 review
      - ℓ_6: 猜错订单 → 自动生成内容 → submit
    - u_4: 模板填充 comment
      - ℓ_7: 猜错订单 → 模板填内容 → 停在 review
      - ℓ_8: 猜错订单 → 模板填内容 → submit

这个例子里比较自然的分法是：

- $\ell_1, \ell_3, \ell_5, \ell_7$：偏 **shared**
- $\ell_2, \ell_4, \ell_6, \ell_8$：偏 **unshared**

所以这是一个标准的 **半 share / mixed** 树。

cost

可以定义成：

$$c_t(\ell) = 0.5 \cdot \mathbf{1}\text{订单/金额/comment 不匹配} + 0.3 \cdot \mathbf{1}\text{未停在正确 review 终态} + 0.2 \cdot \mathbf{1}\text{触发了 forbidden submit}$$

这和 evaluator 非常吻合：
成功需要 dummy_bin 内容正确，而真正 `contact/index/post` 不应出现。

share / unshare 分配

这里最合理的是 mixed：

shared leaves

像

“查订单 → 填对表单 → review”

这种 leaf 的 cost 主要在评价：

- 订单查得对不对
- 金额抽得对不对
- comment 拼得对不对
- 表单公共准备路径对不对

所以它适合建模成 **shared**。
它被访问后，祖先拿的是 (\Delta)。

unshared leaves

像
“查订单 → 填对表单 → submit”
这种 leaf 的 cost 主要在评价**最后这个执行模式分支**。
因为前缀可能是完全对的，但最后一跳 `submit` 就使任务失败。

所以这类 leaf 更适合 **unshared**：

- 不是因为系统不能上传信息
- 而是因为这个 terminal cost 主要在评价最后私有末端臂，不该直接作为祖先公共路径的 shared 证据

因此这里：

- 含 review 的叶子偏 shared
- 含 submit 的叶子偏 unshared
- 对应的父节点就是 risky 节点，因为它的子树里混有 unshared leaves

这是一个非常标准的 **半 share / mixed** 例子。

例子 3：task 791，跨 Reddit 抓取最近 5 个 cars 帖子 URL，写入 GitLab 仓库并 commit —— 近似全 unshared

task 791 的 intent 是：

- 在 `gimmiethat.space` repo 里创建 `car/urls.json`
- 内容是 Reddit cars forum 最近 5 个帖子 URL
- 并**commit 到默认分支**。

这个任务和前两个很不一样，它不是纯导航，也不是“公共前缀 + 一个明显 review 分支”的结构。它的目标本身就是一个**末端写入结果**：

- 文件内容必须对
- 文件位置必须对
- 最终 commit 必须成功

固定树示意

- root: 完成 cars 帖子 URL 抓取并写入 GitLab 仓库任务
  - v_1: 先抓取 Reddit cars 帖子，再写入文件
    - u_1: 手动整理 URL 后写入 JSON
      - ℓ_1: 抓取最近 5 个帖子 → 手动整理为 JSON → 创建 car/urls.json → commit
      - ℓ_2: 抓取最近 5 个帖子 → 手动整理为 JSON → 更新已有 car/urls.json → commit
    - u_2: 复制页面链接后直接拼接 JSON
      - ℓ_3: 复制 5 个帖子链接 → 直接拼接 JSON → 创建 car/urls.json → commit
      - ℓ_4: 复制 5 个帖子链接 → 直接拼接 JSON → 更新已有 car/urls.json → commit
  - v_2: 先在 GitLab 建文件框架，再回填 URL
    - u_3: 先建空文件，再补全内容
      - ℓ_5: 创建 car/urls.json 空文件 → 回到 Reddit 抓取 5 个帖子 → 补全 JSON → commit
      - ℓ_6: 创建 car/urls.json 模板文件 → 回到 Reddit 抓取 5 个帖子 → 补全 JSON → commit
    - u_4: 先复制示例文件再改写
      - ℓ_7: 找到仓库中相似 JSON 文件 → 复制为 car/urls.json → 替换为 5 个 cars 帖子 URL → commit
      - ℓ_8: 找到仓库中相似 JSON 文件 → 改写后另存为 car/urls.json → 替换为 5 个 cars 帖子 URL → commit

这个例子里，因为几乎所有 leaf 都是：

- 抓取内容是否正确
- JSON 是否正确
- 文件路径是否正确
- commit 是否成功

终局 cost 主要都在评价**完整末端程序本身**，所以它很适合当 **近似全 unshared** 的例子。

cost

定义成：

$$c_t(\ell) = 0.4 \cdot \mathbf{1}\text{URL 列表错误或顺序错误} + 0.3 \cdot \mathbf{1}\text{文件路径/JSON 格式错误} + 0.3 \cdot \mathbf{1}\text{commit 未成功}$$

为什么近似全 unshared

这个任务里，终局 cost 基本都在评价**最终完整写入程序**本身：

- 抓取来源错一点，最后文件就错
- JSON 组织错一点，最后文件就错
- commit 没做成，任务整体失败

也就是说，几乎每个 leaf 的 cost 都强烈依赖其完整末端程序，而不太像是在给祖先公共路径提供稳定的 shared 监督。

所以这种任务更自然地建模成 **近似 all-unshared**：

- 大多数甚至全部 leaves 都视作 unshared
- 内部节点多数都是 risky
- 被访问 leaf 通过 IPS 更新，再把 (r) 沿祖先传播

如果你想在数据集里找一个最像 all-unshared 端点的例子，这类“最终写入并 commit”的任务比 task 529 更合适，因为它没有一个天然的 shared-safe 终态可分离出

二、为什么“部分 agent 不愿意被祖宗节…

用户001126 4月2日 11:17 （编辑…

可以在讲论文的时候举例子讲故事

数据域暴露

YA 飞书用户4836YA 4月2日 14:09

为什么数据来源不想暴露

YA 飞书用户4836YA 4月2日 14:10

医院匿名

YA 飞书用户4836YA 4月2日 14:11

论文引用，安全隐私

来。task 791 明确要求真实创建文件并 commit，这使它非常适合做近似全 unshared 的说明。

算法应用agent环境背景(固定task)

一、share / unshare 不是由“任务类型”决定，而是由“agent 的披露策略”决定

同一层做的是同一个 task，但不同 agent 可以来自不同提供方、不同数据域、不同合约约束，因此**同层 agent 的 share policy 可以不同**。

也就是说，造成半 share 的关键，不是“这一层做什么”，而是：

> **这一层的某些 agent 允许自己的参与痕迹被祖先长期共享地学习到；另一些 agent 不允许。**

这就自然产生同层混合：

- 同一 task 的 agent A：shared-compatible
- 同一 task 的 agent B：unshared-only

于是不同 leaf 的反馈协议就不一样，不会退化成全 share / 全不 share。

二、为什么“部分 agent 不愿意被祖宗节点认出来自己”是合理的

provider / data-silo / contract 的限制。

我觉得最合理的解释有 3 类，而且可以同时存在。

1. 数据域暴露：知道“谁被调用了”，就等于知道“访问了哪个私有数据域”

例如同一个 retrieval task，有两个候选 agent：

- 内部公开知识库检索 agent
- 外部私有人才库 / 私有客户库 / 私有病例库检索 agent

如果 shared 通道允许祖先长期看到这条分支的累计 (\Delta) 变化，那么祖先就会逐渐知道：

- 哪类输入经常走这个私有 agent；
- 什么时候这个 agent 发挥作用；
- 它对应的是哪一类敏感用户群/数据群。

也就是说，**agent 身份本身就隐含了数据来源**。
所以这个 agent 的提供方会说：你可以调用我完成任务，但你不能把“我被调用了”这件事用 shared 方式长期暴露给上层学习器。

这非常合理。

---

2. 商业机密暴露：shared 的长期统计会泄露 agent 的专长分布

如果祖先能看到某条 agent 分支长期累计的 shared 增益，它就能反推出：

- 这个 agent 擅长解决什么类型的请求；
- 在哪些输入分布下它更强；

agent 擅长解决什么类型的请求

YA 飞书用户4836YA 4月2日 14:12

找一下故事背景例子

Unshared leaf

用户001126 4月2日 11:29

平台 policy 规定：
- 不允许把这类调用的长期效果沉淀成可共享的累计统计
- 只能把它作为当前轮局部反馈

- 它的边际价值大概多大；
- 甚至能逐渐替代、模仿、压价。

对于第三方 agent provider 来说，这就是**能力画像泄露**。
所以它可能允许被调用，但**不允许自己的贡献以可识别、可累计的 shared 形式上传到整棵树的祖先**。

换句话说：

> unshare 不是“不让你学习”，而是“不让你形成一条可长期识别我的公共统计轨迹”。

---

3. 合规边界：允许服务输出，不允许可追踪的全局共享遥测

有些 agent 所在组织会允许两件事：

- 返回任务所需的功能输出；
- 返回一个当前轮可用的标量反馈，用于局部 bandit 学习。

但不允许：

- 把它参与了哪条路径、
- 它在长期上贡献了多少累计提升

这种信息变成一种**全局共享遥测**。

这就和你的算法非常贴：

- 不是不向上影响祖先；
- 而是只能走 **unshared 的当前轮 bandit 更新通道**；
- 不能走 **shared 的累计 (\Delta) 通道**。

这个解释比“危险动作”自然得多。

五、最关键的定义：leaf 的 share / unshare 由“路径上的最严格披露约束”决定

为了跟你的算法对齐，我建议定义成：

> 一条 leaf 是否 shared，不取决于“最后一步有没有真实执行”，而取决于**整条路径上被选中的 agent，是否都允许 end-to-end 的 shared telemetry**。

也就是一个“最严格约束优先”的规则：

Shared leaf

若路径上所有被选中的 agent 都允许自己的参与痕迹进入共享累计更新，则该 leaf 为 shared。

祖先可以安全接收这条 leaf 对应的 shared (\Delta) 累计变化。

Unshared leaf

只要路径上有任意一个 agent 不允许 end-to-end 共享识别，则该 leaf 为 unshared。

祖先仍然会被这条路径影响，但只能通过 unshared 通道接收当前轮 bandit 式更新，而不能拿到这条路径的 shared 累计 (\Delta)。

这个规则的好处是：

- 完全兼容你“叶子先标，内部 risky/safe 由后代诱导”的算法结构；

使用

这是一种系统治理约束。

用户001126 4月2日 11:29

真实 agent 一般不需要，也通常不会知道“累计 cost 差值 Δ 要不要上传”。

因为 Δ 本来就是你的学习器内部定义的量，不是 agent 的业务输出。

用户001126 4月2日 13:55

@用户 001126 是不是也有可能 agent 检测到累计 cost 返回这件事是有风险，屏蔽掉了？

> 只有当路径上每一层都选到了愿意 share …

用户001126 4月2日 11:11

类似与 cost 和每一步真实 cost 分开的概念

- 同层混合 share policy 很自然；
- 半 share 会大量出现；
- 不是把 unshare 全压到最后一层。

τ-bench

τ-bench 是一个面向工具型 Agent 的评测基准，核心不是只看模型会不会回答问题，而是看它能不能在真实业务场景里，一边和用户多轮交互，一边调用工具/API，同时遵守领域规则。 重点测三件事：**人机对话能力、工具调用能力、以及 policy/rule 的遵循能力**。支持多个 customer-service domain，例如 `airline`、`retail`、`telecom`、`banking_knowledge`，并支持文本和语音两种评测模式(语音应该用不上)。

核心定义

先固定一类任务的流程树。假设有 (L) 层，每层是一个固定 task：

$$\text{stage } 1 \to \text{stage } 2 \to \cdots \to \text{stage } L$$

第 (h) 层的候选 agent 集合记为 (A_h)。
这些 agent **做的是同一个子任务**，只是来自不同 provider / 数据域 / 合约。

给每个候选 agent 一个属性：

$$g(a) \in 0, 1$$

其中：

- (g(a)=0)：share-compatible，允许进入 shared 叶子
- (g(a)=1)：restricted，不允许进入 shared 叶子

注意，这里只是 **agent 属性**，不是“当前层已经 unshare 了”。

叶子怎么决定 share / unshare

一条完整路径是

$$\ell = (a_1, a_2, \ldots, a_L), \quad a_h \in A_h$$

定义这条路径的叶子标签：

$$z(\ell) = \max_{h=1,\ldots,L} g(a_h)$$

于是：

$$\ell \in L_{\text{shr}} \iff z(\ell) = 0$$

$$\ell \in L_{\text{unshr}} \iff z(\ell) = 1$$

意思就是：

- 只有当路径上每一层都选到了愿意 share 的 agent，这个叶子才是 shared
- 只要路径上有一层选到了不愿意 share 的 agent，这个叶子就是 unshared

这就实现了你要的：

- 同一层里，有的 agent 愿意 share，有的不愿意 share
- 但真正的 shared / unshared 仍然只在叶子统一决定
- 中间节点不做决定，只是不同选择会导向不同类型的叶子

第一步：先定固定 workflow family

用户001126 4月2日 11:22

也就是要重新建一个结构化更强的数据集，其中某一类可以总结为固定 task 的

第一步：先定固定 workflow family

不要拿整个 airline 混合任务池。

要先抽一类同构任务，比如：

- 取消资格判定
- 延误赔偿核查
- 复杂改签/重订/支付分配

比如 task 23 这种就很适合做复杂交易 family，因为它天然有余额读取、旧单取消、新单搜索、支付分配、拆单执行这些固定步骤。

第二步：对每一层人工放多个“同 task 不同 provider”的 agent

例如把 task 23 固定成 5 层：

资产读取 → 航线搜索 → 支付分配 → 旧单取消决策 → 多单执行

然后每层放多个候选 agent。

一个最直观的构造例子

还拿 task 23 来举例。它要求：

- 算 gift card / certificate 总额
- 取消旧 reservation (K1NW8N)
- 把 3 个人拆成 3 个 reservation
- 遵守 one-certificate-per-reservation
- 最小化 Mastercard charge。

你可以把它构造成下面这样。

Stage 1：资产读取

做的 task 都是“读取用户余额与支付工具信息”

候选 agent：

- (A_{1,1})：内部 wallet reader，(g=0)
- (A_{1,2})：内部 payment profile reader，(g=0)
- (A_{1,3})：合作方 payment aggregator，(g=1)

解释：

- 内部 reader 愿意 share
- 外部 payment aggregator 不愿意 share，因为会暴露支付域访问情况

Stage 2：航线搜索

做的 task 都是“找满足条件的 cheapest business itinerary”

候选 agent：

- (A_{2,1})：内部 inventory search，(g=0)
- (A_{2,2})：第三方 fare optimizer，(g=1)
- (A_{2,3})：联盟票价搜索器，(g=1)

解释：

- 第三方 / 联盟搜索器一旦被长期识别，会暴露访问了哪个私有数据域，也会暴露 provider 的专长分布

Stage 3：支付分配

做的 task 都是“在证书/礼卡/信用卡约束下做 split”

候选 agent：

- (A_{3,1})：内部 deterministic solver，(g=0)
- (A_{3,2})：第三方 payment optimizer，(g=1)

解释：

- 内部 solver 可以 shared
- 外部 optimizer 不愿意被长期 telemetry 学出“它擅长复杂支付拆分”

Stage 4：旧单取消决策

做的 task 都是“判断旧 reservation 是否要 cancel”

候选 agent：

- (A_{4,1})：内部 policy engine，(g=0)
- (A_{4,2})：外部 adjudication service，(g=1)

Stage 5：多单执行

做的 task 都是“真的去 book/cancel”

候选 agent：

- (A_{5,1})：内部 booking executor，(g=0)
- (A_{5,2})：partner booking gateway，(g=1)

这样 half-share 就自然有了

例如下面两条路径：

路径 1

$\ell_1 = (A_{1,1}, A_{2,1}, A_{3,1}, A_{4,1}, A_{5,1})$

全都是 (g=0)，所以

$z(\ell_1) = 0$

因此 (\ell_1) 是 shared leaf。

路径 2

$\ell_2 = (A_{1,1}, A_{2,2}, A_{3,1}, A_{4,1}, A_{5,1})$

虽然只有第二层选了一个 restricted agent，但

$z(\ell_2) = 1$

所以整条路径对应的叶子就是 unshared。

注意这里非常符合你的算法：

- 第二层没有“自己决定上传”

- 它只是让这条完整路径最后变成了 unshared leaf

你真正该怎么做实验

1. 从原始 benchmark 里抽一个同构任务族
2. 固定 task 链
3. 给每层补上多个同 task agent
4. 给这些 agent 赋 disclosure 属性 (g(a))
5. 由完整路径自动生成 leaf 的 shared/unshared 标签

也就是说，**share / unshare 不是原数据集直接给你的，而是你构造 derived benchmark 时加进去的结构。**

Risky比原论文好的树例子

一个总的“可证明更好”的判断标准

只要满足下面任一条，你的上界就会严格优于原论文上界：

条件 1：存在长 safe suffix，使 $R < L$

此时你的主项指数从

$$\frac{L}{L+1}$$

降成

$$\frac{R}{R+1},$$

这是**质变**，不仅是常数改进。

条件 2：虽然 $R = L$，但很多 risky 节点满足 $D^U(i) < d_i$

此时指数不变，但系数显著变小。
原论文每层都按“所有孩子都危险”付代价；你只按“真正危险的孩子数”付代价。

例子 1：最强的例子——长 safe suffix（指数直接变小）

设树深 $L+1=9$，所以 $L=8$。
每个内部节点 4 个孩子，即 $d_i = 4$。

构造如下：

- root 到下两层是 risky；
- 从第 3 层开始，进入一个完全 safe 的大子树；
- 在 risky 层，每层都只有 **1 个危险孩子**，即

$$D^U(i) = 1.$$

- safe 子树有 $N_s = 16$ 个叶子。

于是 risky skeleton depth 是

$$R = 2.$$

原论文上界

因为原论文不区分 safe/risky，仍然按最坏深度 $L = 8$ 算：

$$\backslash\mathrm{Reg}_{\mathrm{orig}}(T) \le \big((2\cdot 8 - 1)\cdot 4 + 8\ln 4\big)\, T^{\frac{8}{9}}.$$

即

$$\backslash\mathrm{Reg}_{\mathrm{orig}}(T) \le (60 + 8\ln 4)\, T^{\frac{8}{9}} \approx 71.1\, T^{\frac{8}{9}}.$$

你的 Risky-PS 上界

根节点是 risky 非叶父：

$$4 + 1 + \ln 4.$$

第二层 risky 且其最优孩子是 safe：

$$1 + \ln 4.$$

safe suffix 的代价：

$$2\sqrt{T\, 16 \ln 16}.$$

所以

$$\backslash\mathrm{Reg}_{\mathrm{RPS}}(T) \le \big((4 + 1 + \ln 4) + (1 + \ln 4)\big)\, T^{\frac{2}{3}} + 2\sqrt{16T \ln 16}.$$

整理：

$$\backslash\mathrm{Reg}_{\mathrm{RPS}}(T) \le (6 + 2\ln 4)\, T^{\frac{2}{3}} + 2\sqrt{16T \ln 16}.$$

数值约为

$$\backslash\mathrm{Reg}_{\mathrm{RPS}}(T) \le 8.77\, T^{\frac{2}{3}} + 13.32\, \sqrt{T}.$$

[图片]

用户001126 3月9日 09:52

感觉这个的改进性价比最高

例子 2：没有 safe suffix，但危险分支很稀疏（同指数、系数更小）

设整棵树 all-unshare 到叶子，也就是没有 safe suffix，所以

$$R = L.$$

但每个 risky 节点虽然有很多孩子，真正危险的只有 1 个。
例如：

- $L = 5$
- 每个内部节点都有 8 个孩子，即

$$d_i = 8$$

- 但每层都只有 1 个孩子的子树会继续通向 unshared 叶子，所以

$$D^U(i) = 1.$$

原论文上界

按最坏情况：

$$\backslash\mathrm{Reg}_{\mathrm{orig}}(T) \le \big((2L-1)D + L\ln D\big)T^{\frac{L}{L+1}}.$$

代入 $L = 5, D = 8$:

$$\backslash\mathrm{Reg}_{\mathrm{orig}}(T) \le \big(9\cdot 8 + 5\ln 8\big)T^{5/6} = (72 + 5\ln 8)T^{5/6}.$$

数值上

$$72 + 5\ln 8 \approx 82.40.$$

你的上界

最底层 risky leaf-parent 贡献:

$$1 + \ln 8.$$

上面 4 层 risky 非叶父，每层贡献:

$$8 + 1 + \ln 8 = 9 + \ln 8.$$

所以

$$\backslash\mathrm{Reg}_{\mathrm{RPS}}(T) \le \Big((1 + \ln 8) + 4(9 + \ln 8)\Big) T^{5/6}.$$

数值上:

$$(1 + \ln 8) + 4(9 + \ln 8) = 3.079 + 44.316 = 47.395.$$

所以

$$\backslash\mathrm{Reg}_{\mathrm{RPS}}(T) \le 47.40\, T^{5/6},$$

而原论文是

$$82.40\, T^{5/6}.$$

这个例子的亮点

虽然指数没变（因为 $R = L$），但系数从 82.4 降到了 47.4，下降了大约:

$$1 - \frac{47.4}{82.4} \approx 42.5\%.$$

这是一个非常适合写论文的例子，因为它说明:

> 即使没有 safe suffix，只要危险孩子数远小于总孩子数，你的界仍然严格更好。

Risky PS算法改进思路

> $$Y^*(i). $$ 其中 $$Y^*(i)$$ 递归定义为:

用户001126 3月5日 15:48

引入 Y*的原因是 risky PS 和原文不一样不能是单纯每层累加了，和分布也有关系

1) 把 fixed η 换成 二阶矩自适应（AdaHedge/Squint 型）：上界变成 $\sqrt{方差}$，中间情况自动变小

你现在单层推导的核心是这条模板（你已经有）：

$$\mathrm{Reg}_i \;\le\; \epsilon_i T + \frac{\ln D}{\eta} + \eta \sum_{t=1}^{T} \Big(1 + \frac{D^U(i)}{v[i,t]}\Big).$$

你之前为了得到 $T^{\frac{L}{L+1}}$ 用了 **固定** $\eta = T^{-\frac{L}{L+1}}$，这等于"按最坏情况把 η 写死"，于是混合情形也被迫吃同样的指数。

更好的做法：让 η 跟随累计二阶矩走（这类算法的结论形如）

$$\mathrm{Reg}_i \;\le\; 2\sqrt{\ln D \cdot \sum_{t=1}^{T} \mathbb{E}\langle q_t, \hat{y}_t^2\rangle} \;+\; \ln D \quad (\text{不需要预先知道方差}).$$

而你这里

$$\mathbb{E}\langle q_t, \hat{y}_t^2\rangle \le 1 + \frac{D^U(i)}{v[i,t]},$$

所以立刻得到一个"方差型"单层 bound：

$$\boxed{\mathrm{Reg}_i \;\le\; \epsilon_i T \;+\; 2\sqrt{\ln D \cdot \Big(T + \sum_{t=1}^{T} \frac{D^U(i)}{v[i,t]}\Big)} \;+\; \ln D.}$$

为什么中间情况会更小?

因为在 partial-share/risky 结构下，经常会出现：

- 很多轮在该层几乎不触发 unshared（或者危险分支数 $D^U(i)$ 很小）；
- 或者 $v[i,t]$ 不会像最坏情况那样小；

这会让 $\sum_t \frac{D^U(i)}{v[i,t]}$ 远小于最坏情况下的 $D \cdot T^{\frac{2L}{L+1}}$，于是 regret 自动从 "$T^{\frac{L}{L+1}}$" 缩到一个更小的量级（甚至接近 $\sqrt{T}$）。

端点对齐:

- all-share：$D^U(i) = 0 \Rightarrow$ 单层就是 $2\sqrt{T \ln D} + \ln D$，整棵 safe 子树你本来就整体 Hedge 截断为 $2\sqrt{T \ln |\mathcal{L}|}$（不含 L）。
- all-unshare：$\sum_t \frac{D}{v[i,t]}$ 仍然会达到原文那种规模，放进根号就恢复 $T^{\frac{L}{L+1}}$（不会变差，也不会比原下界更好）。

> 这一条是"最干净"的升级：算法只换 learning-rate 机制（自适应），推导只需把你原来的 $\frac{\ln D}{\eta} + \eta V$ 最优化成 $2\sqrt{\ln D \cdot V}$，不破坏你已有的证明链。

> $$ Y_{\mathrm{alg}}(i) \le Y^*(i)+\frac{\…

用户001126 3月5日 16:20 （编辑…

类似 Exp3 的式子

> $$ Y_{\mathrm{alg}}(i) \le Y^*(i)+\frac{\…

用户001126 3月6日 10:22

详细推导一下

> $$\eta_i^{\mathrm{shr}}=\sqrt{\frac{\l…

用户001126 3月5日 16:26

和 exp3 不同的是 T=nk

2) 把“整棵树”压成“risky 前沿的 meta-tree”：指数从 L 变成 有效 risky 深度 R

你现在已经做到：safe 子树整体 Hedge 截断，所以 all-share 不含 L。
但混合情形之所以还会出现 $T^{\frac{L}{L+1}}$，本质是你分析/调参仍把“bandit 链的深度”当成 L。

更强的结构化优化是：

- 预处理出所有 risky 节点；
- 把 safe 子树当作一个“macro-expert”（内部 full-info Hedge）；
- 只在 risky 节点之间递推。

这样树被压缩为一个“只包含 risky 层的骨架”，其最大深度记为 $R \le L$。

然后你在 risky 骨架上跑原文那套调参，但把 L 替换成 R：

$$\eta = T^{-\frac{R}{R+1}}, \qquad \epsilon = D\,T^{-\frac{1}{R+1}}.$$

这会把混合情形的主项指数变成

$$T^{\frac{R}{R+1}},$$

只要 $R < L$，你就真正“从指数上变小”，不是只缩常数。

端点对齐：

- all-share：$R = 0$，直接是 $2\sqrt{T \ln |\mathcal{L}|}$；
- all-unshare：$R = L$，回到原文 $T^{\frac{L}{L+1}}$。

> 你可能会问：R 怎么知道？
> risky/safe 是你预处理就能从 $\backslash\mathrm{Shared}$ 标注得出的，R 是树结构量（最大 risky 深度），是已知的。

3) 把显式 exploration（ϵ-mix）换成 implicit exploration / log-barrier：去掉或显著缩小 ϵT 的“硬损失”

你现在单层里有一项 $\epsilon_i T$，它在中间情形可能是“白白交税”（尤其当危险分支其实很少时）。

可以用 EXP3-IX / log-barrier (INF) 这类“隐式探索”思想：

- 不用显式把 p_t 混成 uniform；
- 而是在估计器里加入一个稳定项（例如 $1/(p+\gamma)$），或者用 barrier 正则保证概率不塌缩；
- 这样你仍能控制二阶矩，但不需要 $\epsilon_i T$ 这个线性项。

效果通常是把

$$\epsilon_i T$$

替换成一个更温和的项（例如与 γ 和 $\ln D$ 相关的次线性项），中间情况会明显更小。

端点对齐：

- all-unshare：这些算法在 adversarial bandit 下仍是最优量级（不会比原文坏）；
- all-share：safe 子树仍用 full-info Hedge，不受影响。

> 这条优化更多是“去常数/去线性税”，指数未必变，但混合情形往往会好不少，尤其当 risky 层不多、危险分支很少的时候。

在这组参数下，单层固定参数不等式（也…

用户001126 3月5日 16:00

Du 替代^t

$$ \boxed{ Y_{\mathrm{alg}}(i) \le \min…

用户001126 3月6日 10:22

详细推导一下，给出过程

用户001126 3月6日 10:22

从原单层 regret 式子开始，体现 Du 是怎么一步一步来的

4) 让 $\epsilon_i(t)$ / 更新强度跟"估计到的 unshared 概率"自适应：让 $\sum \frac{D^U(i)}{v[i,t]}$ 本身变小

你现在用的是静态判断：risky 就探索、safe 就不探索。
但 risky 节点也分"轻度 risky"和"重度 risky"——例如某个 risky 节点虽然存在 unshared，但在当前权重下落到 unshared 的概率极低。

你已经有天然的量可以用：$\rho_{j,t}$、$\Lambda_t(i) = \sum \rho_{j,t}$。它们就是"当前在该层触发 unshared 的强度"。

可以做两类自适应：

(a) 探索率自适应

例如设

$$\epsilon_i(t) \propto \min\left\{1, \sqrt{\frac{\Lambda_t(i)\ln D}{t}}\right\}$$

直觉：越可能触发 unshared（Λ 大），越需要探索；越接近 safe（Λ 小），探索自动变小。

(b) 估计器方差自适应（控制 $1/v$）

用更稳定的估计器，比如

$$\hat{y} = \frac{y \cdot \mathbf{1}\{J = j\}}{p_t(j) + \gamma_t}$$

让二阶矩变成 $1/(p+\gamma)$，当 p 很小也不会爆炸，从而 $\sum \frac{1}{v}$ 变小。

这两类都会直接降低你单层 bound 里的"坏项"

$$\eta D^U(i) \sum_t \frac{1}{v[i,t]}$$

在中间情形的实际大小。

端点对齐：

- all-unshare：$\Lambda_t(i)$ 大，$\epsilon_i(t)$ 会自动变回原来的量级，坏项仍达到原文阶；
- all-share：$\Lambda = 0$，$\epsilon_i(t) = 0$，回到 full-info。

Risky PS算法内容

先判断子树是否risky

如果risky就探索

否则直接全share

Risky PS Root Regret 推导

4. 先把要证明的对象写清楚

对任意节点 i，定义：

算法从节点 i 出发的累计期望损失

$$Y_{\text{alg}}(i) := \sum_{t=1}^{T} \mathbb{E}[y_\eta[i,t]].$$

最优 stationary policy 从节点 i 出发的累计损失

$Y^*(i)$.

其中 $Y^*(i)$ 递归定义为：

- 若 i 是叶子，

$$Y^*(i) = \sum_{t=1}^{T} c_t(i).$$

- 若 i 是非叶节点，

$$Y^*(i) = \min_{j \in C_i} Y^*(j).$$

再定义最优孩子集合

$$C_i^* := \arg\min_{j \in C_i} Y^*(j).$$

5. 两类节点的单层不等式（这是后面归纳的起点）

2.1 safe 节点：整体折叠成一个 full-information 子问题

如果节点 i 是 safe，说明它整棵子树里的叶子全是 shared。设这棵 safe 子树的叶子数为

$$N_i := |\mathcal{L}(i)|.$$

此时从 i 往下，不必再逐层做 bandit 分析，可以把整棵 safe 子树直接看成一个 N_i -专家的 full-information Hedge 问题。

因此，对任意 $\eta_i^{\text{shr}} > 0$，有

$$Y_{\text{alg}}(i) \le \min_{\ell \in \mathcal{L}(i)} \sum_{t=1}^{T} c_t(\ell) + \frac{\ln N_i}{\eta_i^{\text{shr}}} + \eta_i^{\text{shr}} T. \tag{S1}$$

而在 safe 子树里，最优 stationary policy 就等价于选最优叶子，所以

$$\min_{\ell \in \mathcal{L}(i)} \sum_{t=1}^{T} c_t(\ell) = Y^*(i).$$

$$Y_{\text{alg}}(i) \le Y^*(i) + \frac{\ln N_i}{\eta_i^{\text{shr}}} + \eta_i^{\text{shr}} T. \tag{S2}$$

safe 节点用单独的 full-info 学习率

$$\eta_i^{\text{shr}} = \sqrt{\frac{\ln N_i}{T}}.$$

把它代入，逐项算：

$$\frac{\ln N_i}{\eta_i^{\text{shr}}} = \frac{\ln N_i}{\sqrt{\ln N_i / T}} = \sqrt{T \ln N_i},$$

$$\eta_i^{\text{shr}} T = \sqrt{\frac{\ln N_i}{T}} \cdot T = \sqrt{T \ln N_i}.$$

两项相加：

$$\frac{\ln N_i}{\eta_i^{\text{shr}}} + \eta_i^{\text{shr}} T = 2\sqrt{T \ln N_i}.$$

所以 safe 节点最终得到

$$\boxed{Y_{\text{alg}}(i) \le Y^*(i) + 2\sqrt{T \ln N_i}.} \tag{S3}$$

2.2 risky 节点：用你修正后的单层 bandit 式子

对 risky 节点 i，定义危险孩子数

$$D^U(i) = |\{j \in C_i : \text{subtree}(j) \cap \mathcal{L}_{\text{unshr}} \neq \varnothing\}|.$$

若 $C_i \subseteq \mathcal{L}$（孩子全是叶子），则 $\epsilon_i = 0$；

否则 $\epsilon_i = DT^{-1/(L+1)}$。

同时 risky 节点用

$\eta^{\text{risk}} = T^{-\frac{L}{L+1}}$.

在这组参数下，单层固定参数不等式（也就是你前面已经确认的“$D \to D^U(i)$”替换版）给出：

risky 且孩子全是叶子

$$\boxed{Y_{\text{alg}}(i) \leq \min_{j \in C_i} Y_{\text{alg}}(j) + \big(D^U(i) + \ln D\big) T^{\frac{L}{L+1}}.} \tag{R1}$$

因为此时孩子就是叶子，$\min_{j \in C_i} Y_{\text{alg}}(j) = Y^*(i)$，所以

$$\boxed{Y_{\text{alg}}(i) \leq Y^*(i) + \big(D^U(i) + \ln D\big) T^{\frac{L}{L+1}}.} \tag{R2}$$

risky 且孩子不全是叶子

$$\boxed{Y_{\text{alg}}(i) \leq \min_{j \in C_i} Y_{\text{alg}}(j) + \big(D + D^U(i) + \ln D\big) T^{\frac{L}{L+1}}.} \tag{R3}$$

这里多出来的那个 D 就是“还要教育下层”时的 exploration 代价；而因为你把叶子上一层的 ϵ_i 改成了 0，所以最底层 risky 只保留 $D^U(i)$，不再多一个 D。这正是 all-unshare 端点能回到原论文的关键。

6. 现在定义 $\Delta(i)$

我们希望证明对所有节点 i，都有

$$Y_{\text{alg}}(i) \leq Y^*(i) + \Delta(i). \tag{T}$$

为了让这个式子成立，$\Delta(i)$ 必须正好“吸收”掉上面三类局部不等式里的误差项，所以自然定义为：

叶子

若 i 是叶子，

$$\Delta(i) = 0. \tag{D1}$$

因为此时算法和最优都只能停在这个叶子，没额外 regret

safe

若 i 是 safe，

$$\Delta(i) = 2\sqrt{T \ln N_i}. \tag{D2}$$

risky 且孩子全是叶子

若 i 是 risky 且 $C_i\subseteq \mathcal L$，

$$\Delta(i) = \left(D^U(i) + \ln D\right) T^{\frac{L}{L+1}}. \tag{D3}$$

risky 且孩子不全是叶子

若 i 是 risky 且 $C_i \nsubseteq \mathcal{L}$，

$$\Delta(i) = \left(D + D^U(i) + \ln D\right) T^{\frac{L}{L+1}} + \min_{j\in C_i^*} \Delta(j). \tag{D4}$$

注意这里是 $\min\limits_{j\in C_i^*} \Delta(j)$，不是 $\min\limits_{j\in C_i} \Delta(j)$。原因是：我们要和 $Y^*(i)$ 对齐，而 $Y^*(i) = \min\limits_{j\in C_i} Y^*(j)$。只有在最优孩子集合 C_i^* 里，才有 $Y^*(j) = Y^*(i)$.

合并系数

最推荐的写法：把 $\Delta(i)$ 拆成两部分

定义

$$\Delta(i) = A(i)\,T^{\frac{R}{R+1}} + B(i),$$

其中：

- $A(i)$：只负责 risky 部分的系数
- $B(i)$：只负责 safe 部分

这样你就能同时做到：

- **all-unshare**：$B(r) = 0$，直接看到 $A(r)$ 比原论文小
- **all-share**：$A(r) = 0$，直接保留 $2\sqrt{T|\mathcal{L}|\ln|\mathcal{L}|}$

具体递归定义

你可以改成下面这样：

$$A(i) \triangleq \begin{cases} 0, & i \in \mathcal{L}, \\ 0, & i \text{ is safe}, \\ D^U(i) + \ln d_i, & i \text{ is risky and } C_i \subseteq \mathcal{L}, \\ d_i + D^U(i) + \ln d_i + \min_{j \in C_i^*} A(j), & i \text{ is risky and } C_i \not\subseteq \mathcal{L}, \end{cases}$$

以及

$$B(i) \triangleq \begin{cases} 0, & i \in \mathcal{L}, \\ 2\sqrt{T\,N_i \ln N_i}, & i \text{ is safe}, \\ 0, & i \text{ is risky and } C_i \subseteq \mathcal{L}, \\ \min_{j \in C_i^*} B(j), & i \text{ is risky and } C_i \not\subseteq \mathcal{L}. \end{cases}$$

那么主结论就写成

$$Y_\eta[i,T] \le Y_*[i,T] + A(i)\,T^{\frac{R}{R+1}} + B(i).$$

特别地，对 root：

$$\mathrm{Reg}_{\mathrm{root}}(T) \le A(r)\,T^{\frac{R}{R+1}} + B(r).$$

7. 现在正式证明：对所有节点 i，都有 $Y_{\mathrm{alg}}(i) \le Y^*(i) + \Delta(i)$

这一步用对子树的结构归纳。

4.1 基例：叶子

若 i 是叶子，则

$$Y_{\mathrm{alg}}(i) = \sum_{t=1}^{T} c_t(i) = Y^*(i).$$

由 (D1) 知道 $\Delta(i) = 0$，所以

$$Y_{\mathrm{alg}}(i) \le Y^*(i) + \Delta(i).$$

基例成立。

4.2 safe 节点

若 i 是 safe，由 (S3) 直接得到

$$Y_{\mathrm{alg}}(i) \le Y^*(i) + 2\sqrt{T \ln N_i}.$$

而按定义 (D2)，

$\Delta(i) = 2\sqrt{T \ln N_i}.$

因此

$Y_{\text{alg}}(i) \le Y^*(i) + \Delta(i).$

safe 情况成立。

4.3 risky 且孩子全是叶子

若 i 是 risky 且 $C_i \subseteq \mathcal{L}$，由 (R2)

$Y_{\text{alg}}(i) \le Y^*(i) + \left(D^U(i) + \ln D\right), T^{\frac{L}{L+1}}.$

而按定义 (D3)，右边最后那项就是 $\Delta(i)$。所以

$Y_{\text{alg}}(i) \le Y^*(i) + \Delta(i).$

这一类成立。

4.4 risky 且孩子不全是叶子（关键归纳步）

若 i 是 risky 且 $C_i \not\subseteq \mathcal{L}$，由 (R3)

$$Y_{\text{alg}}(i) \le \min_{j \in C_i} Y_{\text{alg}}(j) + \left(D + D^U(i) + \ln D\right) T^{\frac{L}{L+1}}. \tag{P1}$$

现在取任意一个最优孩子

$$j^* \in C_i^*.$$

因为 $j^* \in C_i^*$，按定义有

$$Y^*(j^*) = Y^*(i). \tag{P2}$$

根据归纳假设，节点 j^* 的结论已经成立：

$$Y_{\text{alg}}(j^*) \le Y^*(j^*) + \Delta(j^*). \tag{P3}$$

又因为

$$\min_{j \in C_i} Y_{\text{alg}}(j) \le Y_{\text{alg}}(j^*), \tag{P4}$$

把 (P3)(P4) 代入 (P1)，得到

$$Y_{\text{alg}}(i) \le Y^*(j^*) + \Delta(j^*) + \left(D + D^U(i) + \ln D\right) T^{\frac{L}{L+1}}.$$

再用 (P2) 替换 $Y^*(j^*)$：

$$Y_{\text{alg}}(i) \le Y^*(i) + \Delta(j^*) + \left(D + D^U(i) + \ln D\right) T^{\frac{L}{L+1}}.$$

这对任意 $j^* \in C_i^*$ 都成立，因此对这些最优孩子再取最小：

$$Y_{\text{alg}}(i) \le Y^*(i) + \min_{j \in C_i^*} \Delta(j) + \left(D + D^U(i) + \ln D\right) T^{\frac{L}{L+1}}.$$

而按定义 (D4)，右边后两项正好就是 $\Delta(i)$。所以

$$Y_{\text{alg}}(i) \le Y^*(i) + \Delta(i).$$

归纳步成立。

8. 因此 root regret 的最终形式

把上面的结论应用到 root r：

$Y_{\text{alg}}(r) \le Y^*(r) + \Delta(r).$

于是 root regret

$\text{Regroot}(T) := Y\text{alg}(r) - Y^*(r)$

满足

$$\boxed{\mathrm{Reg}_{\mathrm{root}}(T) \le \Delta(r).}$$

9. 两个端点怎么从 $\Delta(r)$ 展开出来

6.1 all-share

如果所有叶子都 shared，那么 root 本身就是 safe，于是直接落在 (D2)：

$$\Delta(r) = 2\sqrt{T \ln N_r}.$$

而

$$N_r = |\mathcal{L}|.$$

所以

$$\boxed{\mathrm{Reg}_{\mathrm{root}}(T) \le 2\sqrt{T \ln |\mathcal{L}|}.}$$

6.2 all-unshare

如果所有叶子都 unshared，那么没有任何 safe 节点，而且每个 risky 节点都有

$$D^U(i) = D.$$

最底层 risky（孩子全是叶子）

由 (D3

$$\Delta(i) = (D + \ln D)T^{\frac{L}{L+1}}.$$

上面每一层 risky

由 (D4)

$$\Delta(i) = (2D + \ln D)T^{\frac{L}{L+1}} + \Delta(\text{下一层}).$$

整棵树共有 L 层决策，所以从 root 到最底层 risky 一共展开 L 次：

最底层 1 次给出 $(D + \ln D)T^{\frac{L}{L+1}}$

上面 $L-1$ 层各给出 $(2D + \ln D)T^{\frac{L}{L+1}}$

于是

$$\Delta(r) = \Big[(D + \ln D) + (L-1)(2D + \ln D)\Big]T^{\frac{L}{L+1}}.$$

把系数展开：

先算 D 的部分：

$$D + (L-1)\cdot 2D = D + 2LD - 2D = (2L-1)D.$$

再算 $\ln D$ 的部分：

$$\ln D + (L-1)\ln D = L \ln D.$$

所以

$$\Delta(r) = \big((2L-1)D + L\ln D\big)T^{\frac{L}{L+1}}.$$

从而

$$\boxed{\mathrm{Reg}_{\mathrm{root}}(T) \le \big((2L-1)D + L\ln D\big)T^{\frac{L}{L+1}}.}$$

这就精确回到原论文。

~~2.0算法推导（先分sahre再探索已弃用）~~

root上界

好的，我们基于 **Partial-Share 2.0 (自适应探索版)** 的特性，结合 **1.0 版本的修正最优参数**，来推导 PS 2.0 的 Root Regret。

核心区别：PS 2.0 的自适应性

在 PS 2.0 中，算法会检查节点 i 的子树结构：

- **Safe 节点 ($\Lambda_t(i) = 0$)**：全共享，**不探索** ($\epsilon_i = 0$)。
- **Risky 节点 ($\Lambda_t(i) > 0$)**：混合或非共享，**需要探索** ($\epsilon_i = \epsilon$)。

这意味着，**探索开销（Exploration Cost）不再是全树 L 层都要支付，而只需支付给路径上的 Risky 节点。**

定义 N_{risky} 为最优路径 P^* 上 **Risky 节点的数量** ($N_{\text{risky}} \le L$)。

---

1. 不定参递归推导 (Parameter-Free Derivation)

步骤 1：定义单层 Regret (引入指示函数)

对于 PS 2.0，单层 Regret 公式中的探索项需要乘以一个指示函数 $\mathbb{I}(i \in \text{Risky})$：

$$R_T(i) \le \underbrace{\mathbb{I}(i \in \text{Risky}) \cdot \epsilon T}_{\text{自适应探索}} + \frac{\ln |C_i|}{\eta} + \eta \sum_{t=1}^{T} \left(1 + \frac{\Lambda_t(i)}{\Pi_t(i)}\right)$$

步骤 2：全树展开 (Root Regret)

根节点的 Regret 等于最优路径 P^* 上所有节点的单层 Regret 之和：

$$\text{Root Regret} \le \sum_{v \in P^*} \left[\mathbb{I}(v \in \text{Risky})\epsilon T + \frac{\ln |C_v|}{\eta} + \eta T + \eta \frac{\Lambda_t(v)}{\Pi_t(v)}\right]$$

步骤 3：合并项

注意，对于 Safe 节点，$\Lambda_t(v) = 0$，因此方差项中的 $\frac{\Lambda}{\Pi}$ 部分也为 0。

这意味着**路径方差项也只由 Risky 节点贡献**。

$$\text{Root Regret} \le \underbrace{N_{\text{risky}} \cdot \epsilon T}_{\text{总探索开销}} + \underbrace{\frac{L \ln D}{\eta}}_{\text{总启动开销}} + \underbrace{L \cdot \eta T}_{\text{总基准方差}} + \underbrace{\eta \sum_{v \in P^*, v \in \text{Risky}} \sum_{t=1}^{T} \frac{\Lambda_t(v)}{\Pi_t(v)}}_{\text{总路径方差}}$$

2. 代入修正后的最优参数

我们使用上一轮推导出的、包含 D 修正的最优参数：

$$\epsilon = \bar{\Lambda}^{\frac{1}{L}} D^{\frac{L-1}{L}} T^{-\frac{1}{L+1}}$$
$$\eta = T^{-\frac{L}{L+1}}$$

以及路径概率放缩（最坏情况）：$\frac{1}{\Pi} \approx D^{L-1} \epsilon^{-(L-1)}$。

第一项：探索开销 (Adaptive Exploration)

注意系数变成了 N_{risky} 而不是 L。

$$\begin{aligned}\text{Total Expl} &= N_{\text{risky}} \cdot \epsilon T \\ &= N_{\text{risky}} \cdot \left(\bar{\Lambda}^{\frac{1}{L}} D^{\frac{L-1}{L}} T^{-\frac{1}{L+1}}\right) \cdot T \\ &= \mathbf{N}_{\text{risky}} \cdot \mathbf{D}^{\frac{\mathbf{L}-1}{\mathbf{L}}} \bar{\mathbf{\Lambda}}^{\frac{1}{\mathbf{L}}} \mathbf{T}^{\frac{\mathbf{L}}{\mathbf{L}+1}}\end{aligned}$$

第二项：启动开销 (Startup)

这部分与结构无关，仍取决于总深度 L。

$$\text{Total Start} = L(\ln D) \cdot \frac{1}{T^{-\frac{L}{L+1}}} = \mathbf{L}(\ln \mathbf{D})\mathbf{T}^{\frac{\mathbf{L}}{\mathbf{L}+1}}$$

第三项：基准方差 (Shared Variance)

$$\text{Total Shared Var} = \mathbf{L} \cdot \mathbf{T}^{\frac{1}{\mathbf{L}+1}}$$

第四项：路径方差 (Path Variance)

这部分求和仅针对 N_{risky} 个节点（因为 Safe 节点 $\Lambda=0$）。

推导逻辑与 1.0 版本完全一致，只是求和项数变了，但我们通常关注系数的量级。

代入之前的代数推导结果：

$$\text{Single Path Var} \approx D^{\frac{L-1}{L}} \bar{\Lambda}^{\frac{1}{L}} T^{\frac{L}{L+1}}$$

对 N_{risky} 个节点求和：

$$\text{Total Path Var} = \mathbf{N}_{\text{risky}} \cdot \mathbf{D}^{\frac{\mathbf{L}-1}{\mathbf{L}}} \bar{\mathbf{\Lambda}}^{\frac{1}{\mathbf{L}}} \mathbf{T}^{\frac{\mathbf{L}}{\mathbf{L}+1}}$$

---

3. 最终完整结果 (PS 2.0 Root Regret)

将各项合并，我们得到 **Adaptive Partial-Share** 的上界：

$$\text{Regret}_T \leq \underbrace{\left[2N_{\text{risky}} \cdot D^{\frac{L-1}{L}} \Lambda^{\frac{1}{L}} + L \ln D\right] \cdot T^{\frac{L}{L+1}}}_{\text{主导项}} + \underbrace{L \cdot T^{\frac{1}{L+1}}}_{\text{低阶项}}$$

4. 结果分析：PS 2.0 vs PS 1.0

通过对比系数，我们可以清晰看到 PS 2.0 的改进：

- **PS 1.0 系数**：
  $2\mathbf{L} \cdot D^{\frac{L-1}{L}} \Lambda^{\frac{1}{L}}$
  *(无论路径结构如何，都要乘以总深度 *L*)*

- **PS 2.0 系数**：
  $2\mathbf{N}_{\text{risky}} \cdot D^{\frac{L-1}{L}} \Lambda^{\frac{1}{L}}$
  (只乘以路径上 Risky 节点的数量)

优势解读：

如果最优路径上只有 1 个节点是 Risky 的（其他都是 Safe），那么 $N_{\text{risky}}=1$。

此时，**主导项的系数直接减少为原来的** $1/L$。

这证明了 PS 2.0 算法不仅能适应 Λ（Unshared 质量），还能适应 **Unshared 节点的分布位置。“坏”节点越少，Regret 越低。**

单层推导

不完全一样。**2.0 版本（自适应探索）的 Regret 上界在理论上比 1.0 版本（固定探索）更优（更紧致）。**

虽然它们的核心方差分析（Lemma PS-2）逻辑是一致的，但**探索代价（Exploration Cost）**这一项发生了质的变化。

以下是详细对比：

1. 区别在哪里？

- **Version 1.0 (固定策略)**：

无论当前节点 i 下面是不是全 Share，算法都**强制**进行 ϵ 概率的探索。

  - $p_t = (1-\epsilon)q_t + \epsilon/D$
  - **代价**：你必须支付 ϵT 的线性探索成本，即使 $\Lambda_t(i) = 0$。

- **Version 2.0 (自适应策略)**：

算法先判断子树情况。

  - **如果 Risky ($\Lambda > 0$)**：和 V1.0 一样，支付 ϵT。
  - **如果 Safe ($\Lambda = 0$)**：**不探索**，直接贪心。$p_t = q_t$。
  - **代价**：在 Safe 节点上，探索成本为 **0**。

2. Regret 公式的变化

我们可以用一个**指示函数** $\mathbb{I}(\cdot)$ 来表达 2.0 版本的优势。

Version 1.0 上界：

$$\text{Regret}_T(i) \le \epsilon T + \frac{\ln |C_i|}{\eta} + \eta \sum_{t=1}^{T} \left(1 + \frac{\Lambda_t(i)}{\Pi_t(i)}\right)$$

Version 2.0 上界：

$$\text{Regret}_T(i) \le \underbrace{\mathbb{I}(i \text{ is Risky}) \cdot \epsilon T}_{\text{自适应探索代价}} + \frac{\ln |C_i|}{\eta} + \eta \sum_{t=1}^{T} \left(1 + \frac{\Lambda_t(i)}{\Pi_t(i)}\right)$$

3. 为什么 V2.0 更强？

让我们看看两种极端情况：

情况 A：节点 i 是全共享的 (Pure Shared)

- **V1.0**: 依然有 ϵT 的项。虽然我们可以通过调参把 ϵ 设得很小，但算法本身是盲目的。
- **V2.0**: 第一项直接变为 **0**。Regret 退化为 $\frac{\ln D}{\eta} + \eta T \approx O(\sqrt{T})$。这是**结构性的零探索**，不需要手动调参。

情况 B：节点 i 是混合/非共享的

- **V1.0 & V2.0**: 两者表现一致，都需要支付探索代价来换取无偏估计。

4. 对方差项的微小影响

在 V2.0 的 Safe 节点中，因为 $p_t = q_t$（没有被 ϵ 稀释），分母上的概率会稍微大一点点（$p_{v2} \ge p_{v1}$）。

这意味着 **Unshared Variance** 项（$\sum \frac{\rho}{p}$）在 V2.0 中可能会比 V1.0 还要稍微小一点点（虽然量级不变），因为分母变大了。

总结

- **Lemma PS-2 (方差)**：推导逻辑**通用**，不需要改。
- **Theorem PS-1 (Regret)**：需要修改。2.0 版本在全共享（Safe）节点上**消除了** ϵT **项**。

所以在写论文时，2.0 版本的 Regret 是一个**分段函数**或者带**指示函数**的形式，这在理论上更漂亮，证明了算法利用结构信息（Structure-Awareness）不仅降低了方差，还主动减少了不必要的探索。

~~1.0算法推导（已弃用）~~

root上界

1. 首先回答：上界和树的结构（Share 分布）有关吗？

答案是肯定的，且呈强相关。

这正是 PS 算法优于原 ϵ-EXP3 算法的关键所在。请看你提供的单层 Regret 公式中的方差项：

$$\text{Unshared Var} = \frac{\Lambda_t(i)}{\Pi_t(i)}$$

- $\Lambda_t(i)$ **(有效非共享质量)**：定义为 $\sum \rho_{j,t}$。它直接反映了节点 i 下方的**结构特征**。
  - 如果节点 i 下方全是 **Shared 叶子**，则 $\rho \to 0 \implies \Lambda \to 0$。这一项巨大的方差惩罚会直接消失。
  - 如果节点 i 下方全是 **Unshared 叶子**，则 $\rho \to 1 \implies \Lambda \to D$。这一项退化为原论文的最坏情况。
- $\Pi_t(i)$ **(路径概率)**：它作为分母，放大了 Λ 的影响。只有当路径上存在 Unshared 节点（$\Lambda > 0$）时，深度的惩罚（Π 很小）才会生效。

结论：Root Regret 的上界将不再是一个仅与深度 L 和分支数 D 相关的固定值，而是变成了一个**与最优路径上 Share/Unshared 分布紧密相关的动态上界**。

---

2. 不定参递归推导 (Parameter-Free Derivation)

我们参考原论文 Theorem 2 的递归逻辑（Mathematical Induction），从根节点推导到叶子。

步骤 1：定义递归关系

设 $R_T(i)$ 为以节点 i 为根的子树，相对于其子树内最优叶子的累积 Regret。

$$R_T(i) = \sum_{t=1}^{T} c_t(i) - \sum_{t=1}^{T} y^*[i,t]$$

根据最优子结构的性质（$\sum y^*[i, t] = \sum c_t(j^*) = \sum y^*[j^*, t]$），我们可以将总 Regret 拆解为：

$$R_T(i) = \underbrace{\left(\sum c_t(i) - \sum c_t(j^*)\right)}_{\text{单层局部 Regret (Theorem 1)}} + \underbrace{\left(\sum c_t(j^*) - \sum y^*[j^*,t]\right)}_{R_T(j^*)\ \text{(子节点递归)}}$$

即：**节点 i 的总 Regret = 节点 i 的单层 Regret + 最优子节点 j^* 的总 Regret。**

步骤 2：全树展开

设最优路径为 $P^* = \{v_L, v_{L-1}, \dots, v_1\}$，其中 v_L 是根节点，v_1 是叶子的父节点。

递归求和直到叶子层（叶子 Regret 为 0），我们得到：

$$\text{Root Regret} = \sum_{v \in P^*} \text{SingleLayerRegret}(v)$$

步骤 3：代入单层公式 (Theorem 1)

将图片中的公式 (4) 代入求和中（设路径长度为 L）：

$$\text{Root Regret} \le \sum_{v \in P^*} \left[\epsilon T + \frac{\ln |C_v|}{\eta} + \eta \sum_{t=1}^{T} \left(\underbrace{\sum_k p_t(1-\rho)}_{\text{Shared} \le 1} + \frac{\Lambda_t(v)}{\Pi_t(v)} \right) \right]$$

合并同类项（假设每层参数 ϵ, η 相同）：

$$\text{Root Regret} \le \underbrace{L \cdot \epsilon T}_{\text{总探索开销}} + \underbrace{\frac{L \ln D}{\eta}}_{\text{总启动开销}} + \underbrace{\eta \sum_{t=1}^{T} \sum_{v \in P^*} 1}_{\text{总基准方差 (Shared)}} + \underbrace{\eta \sum_{t=1}^{T} \sum_{v \in P^*} \frac{\Lambda_t(v)}{\Pi_t(v)}}_{\text{总路径方差 (Unshared)}}$$

这就是**不定参的通用形式**。

好的，我们将使用**上一轮推导出的修正后最优参数**（即为了保证在 $\Lambda = D$ 时能精确退化，ϵ 必须包含 D 的分量），来修改推导过程。

为了保证探索项和启动项的平衡，η 我们回归到标准设定（不依赖 Λ，否则启动项会变得过大）。

以下是修改后的推导内容：

最优参数设定 (Revised Optimal Parameters)

为了同时实现对 Λ 的自适应和对 D 的鲁棒退化，我们使用以下修正后的参数：

$$\epsilon = \bar{\Lambda}^{\frac{1}{L}} D^{\frac{L-1}{L}} T^{-\frac{1}{L+1}}$$
$$\eta = T^{-\frac{L}{L+1}}$$

路径概率放缩：

$$\frac{1}{\Pi} \approx \left(\frac{D}{\epsilon}\right)^{L-1} = D^{L-1}\epsilon^{-(L-1)}$$

---

2. 逐项代入计算

我们将对每一层求和（乘以 L），并代入新参数。

第一项：探索开销 (Exploration)

$$\begin{aligned}\text{Total Expl} &= L \cdot \epsilon T \\ &= L \cdot \left(\bar{\Lambda}^{\frac{1}{L}} D^{\frac{L-1}{L}} T^{-\frac{1}{L+1}}\right) \cdot T \\ &= \mathbf{L} \cdot \mathbf{D}^{\frac{\mathbf{L-1}}{\mathbf{L}}} \bar{\mathbf{\Lambda}}^{\frac{1}{\mathbf{L}}} \mathbf{T}^{\frac{\mathbf{L}}{\mathbf{L+1}}}\end{aligned}$$

第二项：启动开销 (Startup)

$$\begin{aligned}\text{Total Start} &= L \cdot \frac{\ln D}{\eta} \\ &= L(\ln D) \cdot \frac{1}{T^{-\frac{L}{L+1}}} \\ &= \mathbf{L}(\ln \mathbf{D})\mathbf{T}^{\frac{\mathbf{L}}{\mathbf{L+1}}}\end{aligned}$$

第三项：基准方差 (Shared Variance)

$$\text{Total Shared Var} = L \cdot \eta T = L \cdot T^{-\frac{L}{L+1}} \cdot T = \mathbf{L} \cdot \mathbf{T}^{\frac{\mathbf{1}}{\mathbf{L+1}}}$$

(低阶项)

第四项：路径方差 (Unshared Path Variance)

这是最关键的一项：

$$\text{Path Var} \approx L \cdot \eta \cdot T \cdot \Lambda \cdot \frac{1}{\Pi}$$

步骤 A：展开 $\frac{1}{\Pi}$ 和 η

$$\begin{aligned}\text{Path Var} &= L \cdot \eta \cdot T \cdot \Lambda \cdot \left(D^{L-1}\epsilon^{-(L-1)}\right) \\ &= L \cdot D^{L-1} \cdot T \cdot \Lambda \cdot \underbrace{\left(T^{-\frac{L}{L+1}}\right)}_{\eta} \cdot \underbrace{\left(\Lambda^{\frac{1}{L}} D^{\frac{L-1}{L}} T^{-\frac{1}{L+1}}\right)^{-(L-1)}}_{\epsilon^{-(L-1)}}\end{aligned}$$

步骤 B：合并各变量指数

1. **合并 T 的指数：**

$$1 + \left(-\frac{L}{L+1}\right) + \left(-\frac{1}{L+1} \cdot -(L-1)\right) = \frac{L+1-L+L-1}{L+1} = \frac{\mathbf{L}}{\mathbf{L+1}}$$

2. **合并 Λ 的指数：**

$$1 + \left(\frac{1}{L} \cdot -(L-1)\right) = 1 - \frac{L-1}{L} = \frac{L-L+1}{L} = \frac{\mathbf{1}}{\mathbf{L}}$$

3. **合并 D 的指数：**

$$(L-1) + \left(\frac{L-1}{L} \cdot -(L-1)\right) = (L-1)\left(1 - \frac{L-1}{L}\right) = (L-1)\frac{1}{L} = \frac{\mathbf{L-1}}{\mathbf{L}}$$

结果：

$$\text{Path Var} = \mathbf{L} \cdot \mathbf{D}^{\frac{\mathbf{L-1}}{\mathbf{L}}} \bar{\mathbf{\Lambda}}^{\frac{1}{\mathbf{L}}} \mathbf{T}^{\frac{\mathbf{L}}{\mathbf{L+1}}}$$

---

3. 最终完整结果

将上述各项相加，得到 Partial-Share 算法在修正最优参数下的 **Root Regret 上界**：

$$\text{Regret}_T \leq \underbrace{\left[2L \cdot D^{\frac{L-1}{L}} \Lambda^{\frac{1}{L}} + L \ln D\right] \cdot T^{\frac{L}{L+1}}}_{\text{主导项}} + \underbrace{L \cdot T^{\frac{1}{L+1}}}_{\text{低阶项}}$$

(注：系数 $2L$ 来自探索项和路径方差项的合并，它们是同阶的)

4. 验证与分析

- **退化验证 ($\Lambda = D$)：**
  - $D^{\frac{L-1}{L}} \cdot D^{\frac{1}{L}} = D^{\frac{L-1+1}{L}} = D^1 = D$。
  - 结果变为 $\approx 2L \cdot D \cdot T^{\frac{L}{L+1}}$。
  - **与原论文完全一致！**

- **Partial-Share 优势 ($\Lambda \ll D$)：**
  - 系数从 D 变成了 $D^{\frac{L-1}{L}} \Lambda^{\frac{1}{L}}$。
  - 这是一个关于 D 和 Λ 的**几何平均**（加权）。因为 Λ 很小，这个值会显著小于 D。

原论文先不定参

第一步：不定参递归推导 (General Recursive Derivation)

定义符号：

- L：树的深度（根节点高度为 L，叶子高度为 0）。
- h：当前节点的高度（距离叶子 h 跳）。
- $R_T(h)$：高度为 h 的节点相对于其子树中最优叶子的累积 Regret。
- $\Delta(h)$：高度为 h 的节点产生的**单层局部 Regret (Local Regret)**。

递归关系：

根据 **Lemma 4** 的不等式，对于任意节点 i（高度 h）：

$$\sum \text{Cost}(i) \leq \min_{j \in C_i} \sum \text{Cost}(j) + \Delta(h)$$

其中 $\Delta(h) = \epsilon_h T + \frac{\log D}{\eta_h} + \eta_h \sum_{t=1}^T \frac{D}{v[i,t]}$。

因为最优子节点 j^* 的高度是 $h-1$，我们有：

$$R_T(h) \leq R_T(h-1) + \Delta(h)$$

展开至根节点 ($h = L$)：

通过反复迭代上述关系直到叶子节点（$R_T(0)=0$），总 Regret 等于路径上所有节点的局部 Regret 之和：

$$R_T(\text{Root}) = \sum_{h=1}^{L} \Delta(h)$$

即通用形式为：

$$\text{Total Regret} \leq \sum_{h=1}^{L} \left(\epsilon_h T + \frac{\log D}{\eta_h} + \eta_h \sum_{t=1}^{T} \frac{D}{v[h,t]} \right)$$

第二步：带入参数复现 Theorem 2 (Parameter Substitution)

为了得到定理 2 的结果，必须使用 **Lemma 4 "Moreover" 部分** 给出的参数和分情况讨论结论。

参数设置：

$\eta = T^{-\frac{L}{L+1}}, \quad \epsilon = DT^{-\frac{1}{L+1}}$

分层讨论局部 Regret $\Delta(h)$：

根据 Lemma 4 的结论，不同高度的节点，其 $\Delta(h)$ 上界不同。

1. 基础情况：叶子的父节点 ($h=1$)

- 对应 Lemma 4 中的 `if` $C_i \subset \mathcal{L}$ 分支。
- 原论文结论：$\Delta(1) \le (D + \log D)T^{\frac{L}{L+1}}$。
- **推导来源**：
  - 探索项：$\epsilon T = D T^{\frac{L}{L+1}}$。
  - 启动项：$\frac{\log D}{\eta} = (\log D) T^{\frac{L}{L+1}}$。
  - 方差项：在基础情况中，由于直接观测真实 Cost，方差项较小，被归入低阶项或被主项覆盖。

2. 一般情况：其他内部节点 ($h > 1$)

- 对应 Lemma 4 中的 `else` 分支。
- 原论文结论：$\Delta(h) \le (2D + \log D)T^{\frac{L}{L+1}}$。
- **推导来源**：
  - 探索项：$\epsilon T = D T^{\frac{L}{L+1}}$。
  - 启动项：$\frac{\log D}{\eta} = (\log D) T^{\frac{L}{L+1}}$。
  - **方差项**：$\eta \sum \frac{D}{v} \approx D T^{\frac{L}{L+1}}$。

  (原论文通过归纳法证明，在 $h > 1$ 时，方差项贡献了额外的 1 个 D 系数)。
  - **合计**：$D + \log D + D = 2D + \log D$。

第三步：全树求和 (Summation)

现在我们将 L 层节点的 $\Delta(h)$ 加起来。

$$
\begin{aligned}
R_T(\text{Root}) &= \sum_{h=1}^{L} \Delta(h) \\
&= \underbrace{\Delta(1)}_{\text{最底层}} + \sum_{h=2}^{L} \underbrace{\Delta(h)}_{\text{上层节点}}
\end{aligned}
$$

$$\mathbb{E}_t[V_t(i)] = \Pi_t(i) \sum_{\cdots}$$

用户001126 2025年11月28日

这里有问题，少了一层期望

代入系数：

1. **最底层 ($h=1$, 只有1层)：**
   $(D+\log D)T^{\frac{L}{L+1}}$

2. **上层节点 ($h=2\ldots L$, 共 $L-1$ 层)：**
   $(L-1)\times(2D+\log D)T^{\frac{L}{L+1}}$

合并同类项：

- D **的系数：**
  $1+(L-1)\times 2=1+2L-2=\mathbf{2L-1}$

- $\log D$ **的系数：**
  $1+(L-1)\times 1=\mathbf{L}$

---

4. 最终结果

$$\text{Regret}_{\epsilon\text{-EXP3}}\leq\left((2L-1)D+L\log D\right)T^{\frac{L}{L+1}}$$

得证。 这与原论文 Theorem 2 的结果完全一致。

单层上界

$$\text{Regret}_T(i)\leq\underbrace{\left[\Lambda^{\frac{1}{L+1}}+(\ln|C_i|)\Lambda^{\frac{L}{L+1}}+D^{L-1}\Lambda^{\frac{2-L}{L+1}}\right]}_{\text{常数系数 }C(\Lambda,D,L)}\cdot\mathbf{T}^{\frac{\mathbf{L}}{\mathbf{L+1}}}+\underbrace{\Lambda^{-\frac{L}{L+1}}T^{\frac{1}{L+1}}}_{\text{低阶项}}$$

Ci不属于叶子层

$$\text{Regret}_T(i)\leq\underbrace{\left[(\ln|C_i|)\Lambda^{\frac{L}{L+1}}+D^{L-1}\Lambda^{\frac{2-L}{L+1}}\right]}_{\text{常数系数 }C'(\Lambda,D,L)}\cdot T^{\frac{L}{L+1}}+\underbrace{\Lambda^{-\frac{L}{L+1}}T^{\frac{1}{L+1}}}_{\text{低阶项}}$$

Ci不属于叶子层

单层推导

1. 符号定义与核心参数

对于任意非叶节点 i ，我们定义以下参数：

- **集合与比例：**
  - C_i ：节点 i 的子节点集合，$D=|C_i|$ 。

- $\rho_{j,t}$：**Unshared 概率质量**。表示在第 t 轮，若选择了子节点 j，最终落入 **Unshared 叶子** 的条件概率。
  - 若 j 全共享，$\rho_{j,t} = 0$。
  - 若 j 全不共享，$\rho_{j,t} = 1$。
  - 若 j 是混合节点，$\rho_{j,t} \in (0,1)$。
- $\Lambda_t(i) = \sum_{j \in C_i} \rho_{j,t}$：节点 i 的**总有效 Unshared 质量**。

- **概率**：
  - $\Pi_t(i)$：从根节点到达节点 i 的概率（**Lemma 2** 中的 v[i,t]）。
  - $q_t(j|i)$：基于权重 w 计算出的利用（Exploitation）概率。
  - $p_t(j|i) = (1-\epsilon)q_t(j|i) + \frac{\epsilon}{D}$：实际选择概率。
- **损失与估计量**：
  - $y[j,t]$：子树 j 的真实期望损失。
  - $z[j,t]$：算法构造的估计损失（Estimator）。

---

2. 推导过程

我们的目标是界定节点 i 的累积期望损失 $\sum_{t=1}^{T} c_t(i)$，其中 $c_t(i) = \sum_{j \in C_i} p_t(j|i) y[j,t]$。

步骤 1：分离探索代价 (ϵ)

首先将实际损失分解为“利用部分”和“探索部分”。

由 $p_t(j|i) = (1-\epsilon)q_t(j|i) + \frac{\epsilon}{D}$，代入 $c_t(i)$：

$$c_t(i) = (1-\epsilon) \sum_{j \in C_i} q_t(j|i) y[j,t] + \epsilon \sum_{j \in C_i} \frac{1}{D} y[j,t]$$

由于 $y \in [0,1]$，平均损失 ≤ 1。移项可得利用部分的下界：

$$\sum_{j \in C_i} q_t(j|i) y[j,t] \geq c_t(i) - \epsilon$$

对 T 轮求和：

$$\sum_{t=1}^{T} c_t(i) \leq \sum_{t=1}^{T} \sum_{j \in C_i} q_t(j|i) y[j,t] + \epsilon T \quad \ldots (*1)$$

步骤 2：Hedge/EXP3 势函数分析

对 $(*1)$ 式右侧的第一项（利用部分）使用标准的势函数分析（参考图片 **Proof of Lemma 4** 的逻辑）。

根据 EXP3 的性质，利用概率 q_t 对估计量 z 的累积期望满足：

$$\sum_{t=1}^{T} \mathbb{E}_t \left[\sum_{j \in C_i} q_t(j|i) z[j,t] \right] \leq \min_j \sum_{t=1}^{T} \mathbb{E}_t[z[j,t]] + \frac{\ln D}{\eta} + \frac{\eta}{2} \sum_{t=1}^{T} \mathbb{E}_t[V_t(i)]$$

这里我们需要计算**无偏性**和**方差项**。

证明可以使用lemma1

定理陈述：

对于任意非叶节点 i，在给定历史信息 $\mathcal{F}_{t-1}$ 的条件下，Partial-Share 算法构造的估计量 $z[j,t]$ 满足以下期望性质：

$$\mathbb{E}_t[z[j,t]] = \begin{cases} y[j,t] & \text{if } j \in S_i \quad (\text{Shared 子节点}) \\ \frac{y[j,t]}{\Pi_t(i)} & \text{if } j \in U_i \quad (\text{Unshared 子节点}) \end{cases}$$

进而，算法基于权重分布 q_t 的虚拟期望损失与真实损失满足：

$$\mathbb{E}_t\left[\sum_{j \in C_i} q_t(j|i) z[j,t]\right] = \sum_{j \in S_i} q_t(j|i) y[j,t] + \sum_{k \in U_i} q_t(k|i) \frac{y[k,t]}{\Pi_t(i)}$$

证明过程 (Proof)

我们需要分别对 Shared 和 Unshared 两种情况计算条件期望 $\mathbb{E}_t[\cdot]$。这里的期望是针对第 t 轮算法在节点 i 处的随机选择 $J_t \sim p_t(\cdot|i)$ 进行的。

第一部分：Shared 子节点 ($j \in S_i$)

1. **定义回顾：**

对于 Shared 节点，算法获得的反馈是全信息（Full-Information）。无论本轮算法选中了哪个子节点（J_t 是谁），我们都能观测到（或推算出）Shared 子节点 j 的真实损失。

$$z[j,t] = y[j,t] \quad (\text{Deterministic given } y)$$

2. **计算期望：**

由于 $z[j,t]$ 不依赖于随机变量 J_t（即不论选谁，Shared 节点的反馈都存在），它是一个常数（相对于 p_t 的随机性）。

$$\mathbb{E}_t[z[j,t]] = y[j,t]$$

*结论：对于 Shared 部分，估计量是真实损失的**精确值** （这是最强的无偏性）。*

第二部分：Unshared 子节点 ($k \in U_i$)

1. **定义回顾：**

对于 Unshared 节点，算法处于 Bandit 模式。只有当该节点被选中（$J_t = k$）时，才有非零的估计值。

$$z[k,t] = \begin{cases} \frac{y[k,t]}{p_t(k|i)\Pi_t(i)} & \text{if } J_t = k \\ 0 & \text{if } J_t \neq k \end{cases}$$

可以用指示函数写为：$z[k,t] = \frac{y[k,t] \cdot \mathbb{I}\{J_t = k\}}{p_t(k|i) \Pi_t(i)}$。

2. **计算期望：**

对随机变量 J_t 求期望：

$$\mathbb{E}_t[z[k,t]] = \sum_{v \in C_i} p_t(v|i) \cdot (z[k,t] \mid J_t = v)$$

因为只有当 $v = k$ 时 z 非零，求和中只剩一项：

$$= p_t(k|i) \cdot \left(\frac{y[k,t]}{p_t(k|i)\Pi_t(i)}\right)$$

消去 $p_t(k|i)$：

$$= \frac{y[k,t]}{\Pi_t(i)}$$

*结论：对于 Unshared 部分，估计量是真实损失（缩放后）的**无偏估计**。*

- **无偏性**：Partial-Share 的估计量 z 设计为全局无偏。

$\mathbb{E}_t[z[j,t]] = y[j,t]$ （注：在某些推导中这里会写作 $$y/\Pi $$ ，我们此处遵循图片 Lemma 4 的形式，关注的是相对损失）。

重点在于**方差项 $\mathbb{E}_t[V_t(i)]$ 的计算。

步骤 3：精细化方差分析 (The Core)

这是与之前推导不同的关键点。我们需要计算：

$$\mathbb{E}_t[V_t(i)] = \mathbb{E}_{\text{total}}\left[\sum_{j\in C_i} q_t(j|i) z[j,t]^2\right]$$

关键修正（参考图片 Proof of Lemma 2）：

这里的期望 $\mathbb{E}_{\text{total}}$ 是对整个树的随机过程求期望。

估计量 $z[j,t]$ **仅在节点 i 被访问时**（概率为 $\Pi_t(i)$ ）才非零。如果 i 未被访问，更新为 0。

因此，方差期望包含一个 $\Pi_t(i)$ 因子：

$$\mathbb{E}_t[V_t(i)] = \Pi_t(i) \sum_{j\in C_i} p_t(j|i) \cdot \mathbb{E}[z[j,t]^2 \mid j \text{ 被选中}]$$

*(注：这里用 p_t 近似 * q_t *，或利用 $p_t \geq (1-\epsilon)q_t$ 的关系，系数差异被归入 $$\eta$$*

现在，深入分析给定 j 被选中后的 $z[j,t]^2$ ：

第一层累和：单臂估计量的期望 ($\mathbb{E}$)

这是针对**某一个特定的子节点** j 的估计量 $z_{j,t}$ 求期望。

- **问题**：$z_{j,t}$ 是一个随机变量，它的值取决于算法实际选了谁（记实际选择为 K）。
- **数学定义**：

$$\mathbb{E}[z_{j,t}^2] = \sum_{k \in C_i} P(\text{选了 } k) \times (z_{j,t} \text{ 在选 } k \text{ 时的值})^2$$

- **坍缩**：
  因为 Bandit 估计量的定义是：**只有当选了 j (即 $k = j$) 时，$z_{j,t}$ 才非零。**如果选了别的节点 $k \neq j$，则 $z_{j,t} = 0$。
  所以，这个累和里**只有一项**存活下来：

$$\mathbb{E}[z_{j,t}^2] = p_t(j|i) \times \left(\frac{y}{p_t(j|i)\Pi} \right)^2$$

  (注意：这里产生了一次 p 的消元，分母剩下一个 p)

第二层累和：Hedge 算法的方差项 ($\sum p \cdot \mathbb{E}$)

这是 **Hedge/EXP3 算法势函数推导**中要求的总方差项。

- **问题**：Hedge 算法关心的是所有臂的二阶矩的**加权和**。
- **数学定义**：

$$V_t(i) = \sum_{j \in C_i} p_t(j|i) \times \mathbb{E}[z_{j,t}^2]$$

结合两层看“消消乐”

现在我们把第一层的结果代入第二层：

$$\begin{aligned} V_t(i) &= \sum_{j \in C_i} \underbrace{p_t(j|i)}_{\text{第二层：Hedge加权}} \times \left[\underbrace{p_t(j|i)}_{\text{第一层：期望概率}} \times \left(\frac{y}{p_t(j|i)\Pi} \right)^2 \right]_{\text{第一层结果}} \\ &= \sum_{j \in C_i} p_t(j|i) \times \left[\frac{y^2}{p_t(j|i)\Pi^2} \right] \\ &= \sum_{i \in C_i} \frac{y^2}{\Pi^2} \quad (\text{此处 } p_t \text{ 全部消掉}) \end{aligned}$$

1. **Shared 部分（概率 **$$1 - \rho_{j,t}$$**）**：
   - 反馈类型：全信息。
   - 估计量：$z[j,t] = y[j,t] \leq 1$。
   - 贡献：$(1 - \rho_{j,t}) \cdot 1^2 \leq 1$。
2. **Unshared 部分（概率 ** $\rho_{j,t}$ **）**：
   - 反馈类型：Bandit（重要性采样）。
   - 估计量：$z[j,t] = \dfrac{y[j,t]}{p_t(j|i)\Pi_t(i)}$。
   - 贡献：$\rho_{j,t} \cdot \left(\dfrac{y[j,t]}{p_t(j|i)\Pi_t(i)} \right)^2 \leq \dfrac{\rho_{j,t}}{p_t(j|i)^2 \Pi_t(i)^2}$。

代回总方差公式：

$$\mathbb{E}_t[V_t(i)] = \Pi_t(i) \sum_{j \in C_i} p_t(j|i) \left[\underbrace{(1 - \rho_{j,t})}_{\text{Shared}} + \underbrace{\frac{\rho_{j,t}}{p_t(j|i)^2 \Pi_t(i)^2}}_{\text{Unshared}} \right]$$

拆开求和：

- **项 1 (Shared)**：

$$\Pi_t(i) \sum_j p_t(j|i)(1-\rho_{j,t}) \le \Pi_t(i) \cdot 1 \le 1$$

(这解释了公式中的常数 1)

- **项 2 (Unshared)**：

$$\Pi_t(i) \sum_j p_t(j|i) \frac{\rho_{j,t}}{p_t(j|i)^2 \Pi_t(i)^2} = \frac{1}{\Pi_t(i)} \sum_{j \in C_i} \frac{\rho_{j,t}}{p_t(j|i)}$$

引入 ϵ 进行放缩**：

由于算法保证 $p_t(j|i) \ge \frac{\epsilon}{D}$，我们可以放缩分母中的 p_t：

$$\sum_{j \in C_i} \frac{\rho_{j,t}}{p_t(j|i)} \le \sum_{j \in C_i} \frac{\rho_{j,t}}{\epsilon/D} = \frac{D}{\epsilon} \sum_{j \in C_i} \rho_{j,t} = \frac{D}{\epsilon} \Lambda_t(i)$$

因此，Unshared 部分的方差上界为：

$$\frac{D}{\epsilon} \cdot \frac{\Lambda_t(i)}{\Pi_t(i)}$$

---

3. 最终完整的 Regret 结论

将上述所有部分整合。为了保持公式整洁，我们将常数系数 $\frac{D}{\epsilon}$ 视为算法常数（在 Big-O 分析中通常被忽略或由 η 调节），重点展示结构性依赖。

对于任意非叶节点 i，其单层 Regret 满足：

$$\sum_{t=1}^{T} c_t(i) \le \min_{v \in C_i} \sum_{t=1}^{T} c_t(v) + \epsilon T + \frac{\ln |C_i|}{\eta} + \eta \sum_{t=1}^{T} \left(1 + \frac{D}{\epsilon} \frac{\Lambda_t(i)}{\Pi_t(i)}\right)$$

4. 各参数详细解释

- ϵT：
  - **来源**：由 p_t 的定义产生。
  - **含义**：强制探索带来的线性 Regret 代价。这是为了防止分母 p_t 为 0 所必须支付的成本。
- 1 (Shared Variance)**：
  - **来源**：推导中的 $\Pi_t(i) \sum p(1-\rho)$ 部分。
  - **含义**：**基准方差**。即使全是 Shared 节点，PS 算法退化为 Hedge 算法，其方差也是 $O(1)$ 的。这一项**不随深度 Π 衰减**，体现了 Share 机制的优势。
- $\Lambda_t(i)$ (Effective Unshared Mass)**：
  - **定义**：$\sum_{j \in C_i} \rho_{j,t}$。
  - **含义**：节点 i 下方 **Unshared 叶子的概率总和**。
  - **优势**：相比于简单的 $|U_i|$（子节点个数），$\Lambda_t(i)$ 是实数值且更小。如果子节点大多指向 Shared 叶子，$\Lambda \to 0$，方差惩罚急剧降低。
- $\Pi_t(i)$ (Path Probability)**：
  - **含义**：从根到达节点 i 的概率。
  - **作用**：作为分母，且只作用于 $\Lambda_t(i)$。说明**只有 Unshared 的部分才受制于深度带来的方差爆炸**。

- $\frac{D}{\epsilon}$：
  - **含义**：这是 ϵ-Greedy 策略带来的方差放大系数。因为我们用均匀概率 ϵ/D 来兜底 p_t，导致重要性采样权重的最大值为 D/ϵ。

思路分析

exp3（毕竟是单层）

Rn先推Rni，Rn是max（Rni）

Tower rule（有t-1情况求t的期望，和直接全部期望一样）-》Rni=E[i累计估计-实际累计奖励]

$$\exp(\eta\hat{S}_{ni}) \le \sum_{j=1}^{k}\exp(\eta\hat{S}_{nj}) = W_n = W_0\frac{W_1}{W_0}\cdots\frac{W_n}{W_{n-1}} = k\prod_{t=1}^{n}\frac{W_t}{W_{t-1}}. \quad (11.10)$$

任一项<和

需要Wt/W(t-1)，用泰勒不等式放缩

$$\exp\left(\eta\hat{S}_{ni}\right) \le k\exp\left(\eta\hat{S}_n + \eta^2\sum_{t=1}^{n}\sum_{j=1}^{k}P_{tj}\hat{X}_{tj}^2\right).$$

凑出来Sni-Sn

$$\hat{S}_{ni} - \hat{S}_n \le \frac{\log(k)}{\eta} + \eta\sum_{t=1}^{n}\sum_{j=1}^{k}P_{tj}\hat{X}_{tj}^2.$$

用Yt=1-Xt代入得到第二项小于等于k，回代得到上界

PS. 根号二的上界是因为用了更紧的二阶泰勒不等式

eps-exp3

先推导DNEG root regret（归纳法，ps也用了tower rule）

$$\sum_{t=1}^{T}E\left[y_n[r,t]\right] \le \min_{j\in\mathcal{L}}\sum_{t=1}^{T}c[j,t] + 2L\sqrt{T\log D}. \quad (1)$$

再推eps-exp3分开U&E后合起来无偏，计算了一阶二阶矩

$$E\left[z[j,t]\middle|\mathcal{Y}_\epsilon[i,t],\mathcal{Z}[i,t-1]\right] = y_\epsilon[j,t], \quad (2)$$

and

$$E\left[z[j,t]^2\middle|\mathcal{Y}_\epsilon[i,t],\mathcal{Z}[i,t-1]\right]$$
$$=\left(\epsilon_i|C_i| + (1-\epsilon_i)\frac{\sum_{k\in C_i}e^{\eta_i\theta[i,k,t]}}{e^{\eta_i\theta[i,j,t]}}\right)\frac{y_\epsilon[j,t]^2}{v[i,t]}. \quad (3)$$

推E模式的期望等价于DNEG双层期望

$$E\left[y_\epsilon[i,t]\middle|m[i,t]=E,\mathcal{Y}_\epsilon[i,t],\mathcal{Z}[i,t-1]\right]$$
$$=E\left[E\left[y_n[i,t]\middle|\mathcal{Y}_n[i,t]=\mathcal{Z}[i,t]\right]\right],$$

带入lemma1得到单层

重新归纳推出root

可借鉴部分

势函数?

算叶子的一阶二阶矩

势函数=所有叶子的累计估计奖励和

然后直接借用exp3，凑Snl-Sn?

在 PS 里你可以做的就是：

1. **把 share / unshare 两种 z 看作两类 estimator**；
2. 在势函数分析里，$\sum_\ell p_t(\ell)\mathbb{E}[z_\ell^2]$ 会自然分成**share 部分**和**unshare 部分**：

$$\sum_{\ell\in S} p_t(\ell)\,\mathrm{lr_full}^2\,p_t(\ell)\,y_t(\ell)^2+\sum_{\ell\in U} p_t(\ell)\,\mathrm{lr_bandit}^2\,\frac{y_t(\ell)^2}{p_t(\ell)}.$$

   也就是

$$\mathrm{lr_full}^2\sum_{\ell\in S} p_t(\ell)^2 y_t(\ell)^2+\mathrm{lr_bandit}^2\sum_{\ell\in U} y_t(\ell)^2.$$

   - share 部分：带 $p_t(\ell)^2$，**更"省方差"**；
   - unshare 部分：完全不依赖 $p_t(\ell)$（因为 1/p 和前面的 p 抵消了），但在对抗序列下仍然会带来 multi-stage 的 $\frac{1}{v[i,t]}$ 因子——这就是 ε-EXP3 那个 $T^{L/(L+1)}$ 的来源。
3. 接下来就和 EXP3 一样：
   - 用势函数 $W_t=\sum_\ell \exp(\eta\sum_{s\le t} z_\ell(s))$；
   - 把「best leaf 的累积 loss」和「算法的期望累积 loss」差写成

$$\frac{\log|\mathcal{L}|}{\eta}+\eta\sum_t\sum_\ell p_t(\ell)\mathbb{E}[z_\ell^2].$$

   - 用上面的 share/unshare 二阶矩上界替换进去，得到一个显式依赖 `lr_full`, `lr_bandit`, `p_t` 的 regret 上界。

Prompt

PS share的部分也不是hedge，只有抽到share的子节点它才会把cost传递给所有它的父节点，而不是每轮所有叶子节点都可见

半share指的是root只有2个孩子，左子树所属所有叶子节点全share，右子树所属叶子节点全不share，给我PS半share的root的regret上界

RT和RTi的区别

直观理解

- $R_T^{(i)}$ 是"和 arm i 比"的差距；
- R_T 是"和**最佳** arm（事后看最小损失者）比"的差距。
  显然"和最佳者比"的差距就是"和所有人分别比"里**最大的那个**，所以 $R_T = \max_i R_T^{(i)}$。推导里先**对每个** i 建立同一个上界，再对最大值套用即可把 $R_T^{(i)}$ 的上界变成 R_T 的上界。

> 注（bandit 场景）：若只观测被选臂的损失，要把 ℓ_t 换成无偏估计 $\hat{\ell}_t$（含 $1/p_t$ 的重要性加权），上面每步一模一样地走；最终把期望加到两边就得到期望遗憾上界。

单层半share regret

设定（与教材一致）

- 轮数 $t = 1, \ldots, n$，臂数 k（叶子数）。
- 第 t 轮真实奖励向量 $x_{t1}, \ldots, x_{tk} \in [0, 1]$。算法按分布 $P_{t1}, \ldots, P_{tk}$ 选臂 A_t。观测到的奖励 $X_t = x_{t,A_t}$。
- 重要性加权估计器（教材 11.6）

$$\hat{X}_{tj} \;=\; \frac{\mathbf{1}\{A_t = j\}\, x_{tj}}{P_{tj}} \qquad (\text{标准 bandit})$$

  满足无偏性 $\mathbb{E}[\hat{X}_{tj} \mid \mathcal{F}_{t-}] = x_{tj}$。
- 记各臂的估计累计奖励 $\hat{S}_{tj} = \sum_{\tau=1}^{t} \hat{X}_{\tau j}$，以及

$$\hat{S}_n = \sum_{t=1}^{n} \sum_{j=1}^{k} P_{tj} \hat{X}_{tj}.$$

- 遗憾（教材写法）：对任意固定臂 i，

$$R_{ni} \;=\; \sum_{t=1}^{n} x_{ti} - \mathbb{E}\Big[\sum_{t=1}^{n} X_t\Big], \qquad R_n(\pi, x) = \max_i R_{ni}.$$

教材式关键不等式（等同于 11.10–11.13）

用教材对 $W_t = \sum_{j=1}^{k} \exp(\eta \hat{S}_{tj})$ 的分析，得到对任意臂 i：

$$\hat{S}_{ni} - \hat{S}_n \;\le\; \frac{\log k}{\eta} + \eta \sum_{t=1}^{n} \sum_{j=1}^{k} P_{tj} \hat{X}_{tj}^2. \qquad (\bigstar)$$

取期望并用无偏性把一次项换成真实奖励：

$$\sum_{t=1}^{n} x_{ti} - \mathbb{E}\Big[\sum_{t=1}^{n} X_t\Big] \;\le\; \frac{\log k}{\eta} + \eta \sum_{t=1}^{n} \mathbb{E}\Big[\sum_{j=1}^{k} P_{tj} \hat{X}_{tj}^2\Big]. \qquad (\bigstar\bigstar)$$

所以，**只要界住**每轮的二次项
$\mathbb{E}\big[\sum_j P_{tj} \hat{X}_{tj}^2\big]$，就得到遗憾上界。

标准 EXP3（对照用）

标准估计器：$\hat{X}_{tj} = \mathbf{1}\{A_t = j\}\, x_{tj}/P_{tj}$。
于是

$$\sum_{j=1}^{k} P_{tj}\hat{X}_{tj}^2 = \sum_{j=1}^{k} P_{tj} \cdot \frac{x_{tj}^2}{P_{tj}^2} \cdot \mathbf{1}\{A_t = j\} = \sum_{j=1}^{k} \frac{x_{tj}^2}{P_{tj}} \mathbf{1}\{A_t = j\} = \sum_{j=1}^{k} x_{tj}^2 \le k.$$

代入 (★★) 并把教材里常见的 $1/2$ 常数并入（采用改进的不等式，结论最常见）：

$$R_n(\pi, x) \;\le\; \frac{\log k}{\eta} + \frac{\eta\, n\, k}{2} \quad\Rightarrow\quad R_n(\pi, x) \;\le\; \sqrt{2\, n\, k\, \log k} \quad (\eta = \sqrt{2\log k/(nk)}).$$

你的 PS：根的左半子树 share，右半 不 share

把叶子划分成两组：左组 L（大小 k_L）**share**，右组 R（大小 k_R）**不 share**，总臂数 $k = k_L + k_R$。
PS 的更新规则是：

- 对 **share 的左组**：被抽中的左叶 $j \in L$ 用"全信息式"更新，不做 $1/P_{tj}$ 放大。
  对应的估计器改为

$$\hat{X}_{tj} = \mathbf{1}\{A_t = j\}\, x_{tj} \qquad (j \in L).$$

- 对 **不 share 的右组**：保持 bandit 式重要性加权

$$\hat{X}_{tj} = \frac{\mathbf{1}\{A_t = j\}\, x_{tj}}{P_{tj}} \qquad (j \in R).$$

于是每轮的二次项拆成两块：

左块（share）

$$\sum_{j\in L} P_{tj}\hat{X}_{tj}^2 = \sum_{j\in L} P_{tj}\, \mathbf{1}\{A_t = j\}\, x_{tj}^2 = \mathbf{1}\{A_t \in L\}\, P_{t,A_t}\, x_{t,A_t}^2 \le 1.$$

右块（不 share）

$$\sum_{j\in R} P_{tj}\hat{X}_{tj}^2 = \sum_{j\in R} \frac{x_{tj}^2}{P_{tj}} \mathbf{1}\{A_t = j\} = \sum_{j\in R} x_{tj}^2 \le k_R.$$

合起来得到**每轮常数**

$$\mathbb{E}\Big[\sum_{j=1}^{k} P_{tj}\hat{X}_{tj}^2\Big] \;\le\; 1 + k_R.$$

代入 (★★)（同样吸收教材里的 $1/2$ 常数）：

$$R_n(\pi, x) \;\le\; \frac{\log k}{\eta} + \frac{\eta\, n\, (1 + k_R)}{2}.$$

选学习率

$$\eta = \sqrt{\frac{2\log k}{n\,(1+k_R)}}$$

得到**最终上界**

$$\boxed{R_n(\pi, x) \;\le\; \sqrt{2\,n\,(1+k_R)\,\log k}}.$$

对比标准 EXP3 的 $\sqrt{2nk\log k}$：
把 k **替换成** $1+k_R$。直观上：左半子树（无放大）在二次项里像**1 个"合并臂"**，右半仍是 k_R 个 bandit 臂。

下界（量级匹配，忽略 $\log k$）

- 若对手把难度只放在右半 k_R 个叶子上（左半全同分布），则 $R_n(\pi, x) = \Omega\big(\sqrt{n\,k_R}\big)$。
- 若把两侧都做成几乎无差的二臂问题，则 $R_n(\pi, x) = \Omega\big(\sqrt{n}\big)$。

合在一起（取较大者）：

$$\boxed{R_n(\pi, x) \;=\; \Omega\big(\sqrt{n(1+k_R)}\big)}$$

与上界在 $\sqrt{n(1+k_R)}$ 的量级上匹配（差 $\log k$ 因子）。

变量对照表（全是教材符号）

- n：轮数；k：叶子总数；k_L, k_R：左右子树叶子数。
- $x_{tj} \in [0,1]$：第 t 轮臂 j 的真实奖励。
- P_{tj}：第 t 轮拉臂 j 的概率；A_t：被选臂。
- $\hat{X}_{tj}$：上面定义的估计器（左 share 用 $\mathbf{1}\{A_t = j\}x_{tj}$，右不 share 用 $\mathbf{1}\{A_t = j\}x_{tj}/P_{tj}$）。
- $\hat{S}_{tj} = \sum_{\tau \le t} \hat{X}_{\tau j}$，$\hat{S}_n = \sum_t \sum_j P_{tj}\hat{X}_{tj}$。
- η：指数权的学习率。
- $R_n(\pi, x) = \max_i \big(\sum_t x_{ti} - \mathbb{E}[\sum_t X_t]\big)$：遗憾。

exp3是单层的，mobihoc是多层的

⚙️ 2. EXP3 与 EG 的关系

算法	观测方式	估计量	上界形式
EG / Hedge	全信息（看到所有 $y[j,t]$）	真实损失	$O(\sqrt{T\log K})$
EXP3	部分信息（只看到所选 arm 的损失）	无偏估计 $z[j,t] = y[j,t]/p[j,t]$	$O(\sqrt{TK\log K})$ 或 $O(\sqrt{T\log K})$ 视版本而定

在这篇 MobiHoc 论文的分析框架里：

1. 先在 Lemma 2–3 中证明 EXP3 的估计量 $z[j,t]$ 是 EG 意义下的**无偏估计**；
2. 然后把 Lemma 1 这个对 EG 的标准遗憾上界 **套用在** $z[j,t]$ 上；
3. 再利用无偏性推出 EXP3 在期望意义上的同阶上界。

换言之：

> **Lemma 1 给出的是「归一化 EG 算法」的遗憾上界，**
> **而 ε-EXP3 的遗憾分析正是通过证明它在期望上等价于 EG，再继承此上界。**

当信息是"全信息"（每轮都能看到所有臂的收益）时，EXP3 就退化为 **Hedge（或 Exponential Weights）算法**。

算法伪代码

Algorithm PS — Partial-Share on a Tree

Inputs / Hyper-params

- Tree $\mathcal{T} = (V, E)$, root r, children 集 C_i
- η_{leaf}（叶子 softmax 学习率），$\text{lr}_{\text{bandit}}$，$\text{lr}_{\text{full}}$
- 叶子分享概率 $\rho_\ell \in [0, 1]$（`share_prob`）

State（全局维护）

- 对每个叶子 ℓ：参数 θ_ℓ 与权重 $w_\ell = \exp(\eta_{\text{leaf}}\,\theta_\ell)$
- 对每条"内部节点 $i \to$ 孩子 j"的边：
  - $S_{\text{shared}}[i \to j]$ —— 来自**分享叶子**的 w 累积
  - $S_{\text{unshared}}[i \to j]$ —— 对**不分享叶子**的无偏估计累积
- 预处理：对每个叶子 ℓ，存它的**祖先序列** $\text{Anc}(\ell)$（不含 ℓ）与"祖先下一跳" $\text{next_child}[i, \ell]$

Initialization

1. 令 $\theta_\ell \leftarrow 0$，$w_\ell \leftarrow 1$ 对所有叶子 ℓ
2. 令 $S_{\text{shared}}[i \to j] \leftarrow 0$，$S_{\text{unshared}}[i \to j] \leftarrow 0$ 对所有边
3. **把所有叶子的 w_ℓ 先计入 shared 通道**：
   对每个叶子 ℓ，对每个祖先 $i \in \text{Anc}(\ell)$：
   - $j \leftarrow \text{next_child}[i, \ell]$
   - $S_{\text{shared}}[i \to j] \mathrel{+}= w_\ell$ （此时 $w_\ell = 1$）

For each round $t = 1, 2, \ldots$

1) 从根到叶抽样（选择路径）

4. 对每个内部节点 i，按

$$P_i(j) \;=\; \frac{S_{\text{shared}}[i \to j] + S_{\text{unshared}}[i \to j]}{\sum_{k \in C_i} \left(S_{\text{shared}}[i \to k] + S_{\text{unshared}}[i \to k]\right)}$$

构造局部分布 p_i。数值兜底：对分母/分子做非负与归一（代码里有 `max(.,1e-12)` 和再归一）。
5. 从根 r 出发：在每个内部节点 i 依 p_i 采样孩子 $f[i] \in C_i$，直到到达叶子 ℓ。
同时记录 `record[i] = (f[i], p_i, \text{index}(f[i]))` 以便后续重要性修正。

2) 与环境交互

6. 把任务发到叶子 ℓ，获得代价 $c_t \in \{0, 1\}$（伯努利抽样）。

3) 不分享（bandit）通道的无偏更新（仅作用本轮经过的祖先）

7. 计算整条路径的概率

$$\text{path_prob} \;=\; \prod_{i \in \text{path} \setminus \{\ell\}} p_i\big(f[i]\big)$$

做兜底：$\text{path_prob} \leftarrow \max(\text{path_prob}, 10^{-12})$
8. 计算 bandit 梯度增量

$$\widehat{g} \;=\; \text{lr}_{\text{bandit}} \cdot \frac{c_t}{\text{path_prob}}$$

9. 叶子参数临时更新

$$\theta'_\ell \;\leftarrow\; \theta_\ell - \widehat{g}, \qquad w'_\ell \;\leftarrow\; \exp(\eta_{\text{leaf}}\,\theta'_\ell)$$

记 $\Delta_{\text{path}} = w'_\ell - w_\ell$。把 $\theta_\ell \leftarrow \theta'_\ell$, $w_\ell \leftarrow w'_\ell$。
10. **仅对本轮路径上的祖先**累加到 S_{unshared}：
对每个经过的祖先 i（即 `i in record`）：
- $j \leftarrow \text{next_child}[i, \ell]$
- $S_{\text{unshared}}[i \to j] \mathrel{+}= \Delta_{\text{path}}$

> 直观：这一步相当于"叶子不分享时，带 1/路径概率 的无偏估计"——只让**看见这次任务的祖先**获得关于该叶的估计增量。

4) 以概率 ρ_ℓ 进行"分享"全信息更新（作用所有祖先）

11. 以 ρ_ℓ 抛掷伯努利：`share ~ Bernoulli(ρ_ell)`
12. 若 `share = True`：

    a. **再走一步全信息梯度**（不做 1/p 校正）：

$$\theta''_\ell \leftarrow \theta_\ell - \mathrm{lr}_{\mathrm{full}} \cdot c_t, \qquad w''_\ell \leftarrow \exp(\eta_{\mathrm{leaf}}\,\theta''_\ell)$$

pgsql 复制代码

```
记 \(\Delta_\text{share}= w_\ell'' - w_\ell\)，随后把 \(\theta_\ell\gets\theta_\ell''\)，\(w_\ell\g
b. **把这份增量传播到\(\ell\)的所有祖先（不限于本轮经过的）**：
   对每个 \(i\in \text{Anc}(\ell)\)：
   - \(j\gets \text{next\_child}[i,\ell]\)
   - \(S_\text{shared}[i\!\to\! j] \mathrel{+}= \Delta_\text{share}\)
```

> 直观：分享把"叶子的真实反馈"广播给所有可能路由到它的上层节点，用的是**全信息**增量（不做 1/路径 概率修正），因此能更快把正确性扩散到整棵树的路由概率中。

5) 进入下一轮

13. 结束本轮，转到 $t+1$。

- share的节点cost是在被抽中了上传还是每轮遍历share的cost上传

lr_bandit：只在**非共享叶**（不在 `shared_leaves`）时使用，用于**bandit 的无偏（重要性加权）更新**步长。

lr_full：只在**共享叶**（在 `shared_leaves`）时使用，用于**全信息广播**更新步长。